



CAM

di Camerino

36

From Multi-Agent Simulation to Causal Multilevel Graphs in Parkinson's Disease

Professor

MERELLI Emanuela

Student

MERMALI Alessandro Massimo

University of Camerino

Master of Computer Science (LM-18)

SCHOOL OF SCIENCES AND TECHNOLOGIES

Distributed Calculus and Coordination - MultiAgent Systems Lab

Academic Year 2025/2026

Declaration on the Use of Artificial Intelligence Tools

During the development of the project and the preparation of this report, artificial intelligence tools (specifically GPT-5.5) were used as support instruments for specific tasks.

The use of these tools was limited to assisting the writing process and improving the clarity, consistency and presentation of the document. The scientific content, methodological choices, interpretation of results, software implementation (with the exception of the previously explicitly declared tasks) and final responsibility for the work remain entirely attributable to the author.

In particular, AI-assisted tools were used for:

- generation of test cases from natural language specification;
- updating pre-written documentation with updated classes and behaviors (e.g., *parameter_guide.md*);
- post-run log scraping, processing and diagnostic analysis;
- legacy code cleaning;
- support in visualization and dashboard scripting;
- improving the English formulation of selected paragraphs;
- proposing possible report structures and section organization;
- helping with LaTeX formatting of tables, formulas, and agent specifications;
- reviewing the terminology and consistency across the document;
- converting markdown documentation into latex-ready text.

AI tools were not used as an autonomous source of scientific authority. All technical claims, code-related descriptions and modelling choices were checked against the implemented project, the configuration files and the relevant course material. The author reviewed, edited and approved the final content of the report.

The software project, simulation model, configuration choices and analysis workflow were developed and validated by the author. Any errors, omissions or inaccuracies remain the responsibility of the author.

Author: Alessandro Massimo Mermali

Date: June 12, 2026

Abstract

Parkinson's disease is a complex neurodegenerative pathology, characterized by phenomena happening on different scales: intracellular alterations, protein aggregation, oxidative stress, overwhelming of clearance mechanisms, neuronal death and inflammatory response from the external environment. The distributed and independent nature of these phenomena makes MultiAgent models particularly suitable for the representation of the dynamic evolution of the system.

The project discussed in this report presents an explorative framework for simulation, analysis and visualization of some mechanisms related to the disease's progression. The system is implemented in Python using Repast4Py and MPI, and it is organized as a distributed multi-agent simulation. The simulation operates on two spatial levels: an extracellular environment, representing the substantia nigra, and multiple local intracellular environments, each associated with a specific neuron.

A relevant contribution consists in the integration between simulation and causal tracking. The framework also logs events and snapshots in a temporal causal graph, subsequently contracted in more synthetic representations: a time-contracted graph (G_1), an agent-clustering contraction graph (G_2) and a topological pattern based graph (G_3). This organization makes it possible to range from macroscopic observations on temporal series to the inspection of event chains.

The present report describes the context, the requirements, the architecture, the agent modeling, the simulation loop, parallelization, logging system, post-processing phase and the Streamlit-backed dashboard interface. The framework must be interpreted as a didactical computational and explorative model: it is not developed for actual diagnosis, clinical use, or quantitative biological prediction.

KEY WORDS: multiagent simulation, Parkinson, Repast4Py, MPI, alpha-synuclein, causality, multilevel graph

Contents

Declaration on the Use of Artificial Intelligence Tools	1
1 Introduction	3
1.1 Motivation	3
1.2 Project goals	3
1.3 Main contributions	3
1.4 Report organization	4
2 Biological background	6
3 Theoretical background	8
3.1 Adaptive Agent Systems	8
3.1.1 Formality	8
3.2 Multilevel graph	9
3.2.1 Multilevel Graph Structure	9
3.2.2 Multilevel Analysis	10
4 Simulation	11
4.1 Agent Specification	11
4.1.1 Ag_{neuron} : Dopaminergic Neuron Agent	11
4.1.2 $Ag_{microglia}$: Microglial Agent	15
4.1.3 Ag_{astro} : Astrocyte Agent	18
4.1.4 Ag_{mito} : Mitochondrial Agent	21
4.1.5 $Ag_{lysosome}$: Lysosomal Agent	27
4.1.6 $Ag_{\alpha syn}$: Alpha-Synuclein Agent	33
4.1.7 Ag_A : Alpha-Synuclein Aggregate Agent	39
4.2 Parameters and Stochasticity	45
4.2.1 Structural Parameters	45
4.2.2 Random Seed and Reproducibility	45
4.2.3 Interpretation of Parameters	46
4.3 Simulation Loop	53
4.3.1 Initialization	53
4.3.2 Distributed Execution	54
5 Post-Run Data Pipelines	55
5.1 Simulation Data Logging	55
5.1.1 Runtime Log Files	56
5.1.2 Semantic Causal Event Logging	56
5.1.3 Spatial Snapshot Logging	58
5.1.4 Initialization Logging	58
5.1.5 Tick Metric Logging	58
5.2 Graph Builder Outputs	59

6	Multilevel Graph Analysis	60
6.1	G0: Temporal Causal Trace	61
6.1.1	G0 Nodes	61
6.1.2	G0 Edges	62
6.2	G1: Time Contraction	63
6.3	G2: Agent/State Clustering	65
6.4	G3: Topological SCC Contraction	66
6.5	Why SCC is Used for G3	68
6.6	Practical Reading Strategy	69
7	Dashboard Interface	70
7.1	Purpose of the Dashboard	70
7.2	Dashboard Organization	70
7.3	Parameter Editing	71
7.4	Simulation Monitoring	71
7.5	Post-run Metrics	71
7.6	Initialization and Spatial Reconstruction	71
7.7	Graph Exploration	71
7.8	Role in the Analysis Workflow	72
8	Conclusions and Future Expansions	73
8.1	Achieved Objectives	73
8.2	Scientific and Methodological Interpretation	74
8.3	Limitations	74
8.4	Future Expansions	75

Chapter 1

Introduction

1.1 Motivation

Parkinson's disease is a meaningful example of a complex biological system. The observable tissue damage emerges from the interaction of processes that occur at different scales and that can interact with each other. Said processes includes loss of protein homeostasis, misfolding and aggregation of alpha-synuclein, mitochondrial function alteration, reduction of lysosomal efficiency, oxidative stress increase, glial activation and progressive impairment of dopaminergic neurons.

An exclusively aggregated description may hide the difference between the simulated individuals and the local chains of interactions. On the contrary, a multiagent simulation makes it possible to explicitly represent autonomous entities, local states, positions, perceptions and actions. The global behavior is not forced directly, but emerges from the repeated execution of microscopic rules.

1.2 Project goals

The objective of the presented project is to provide a software environment where:

1. configure populations and model parameters;
2. execute a distributed simulation through MPI;
3. observe the temporal evolution of global variables;
4. reconstruct the initial state, spatial and stochastic distribution of agents;
5. log causal events and cause-effect relationships;
6. generate causal graphs at different level of abstraction;
7. analyze the results through a CLI and a dashboard Streamlit;

The objective of the model is not to replicate in a faithful way the human pathology, but to study how the previously mentioned local mechanisms can produce global trajectories, and how these trajectories can be explained using explicit causal structures.

1.3 Main contributions

The main software and methodological contributions of the project are:

- a multiagent adaptive model based on the `see` $\rightarrow$ `next` $\rightarrow$ `action` $\rightarrow$ `do` loop;
- a separation between extracellular and intracellular habitats;
- a collective and consistent handling of the alpha-synuclein aggregation;

- a distributed simulation using Repast4Py and MPI;
- global time metrics and reconstruction of the state;
- causal logging with identification of nodes, events and edges;
- multilevel contraction pipeline;
- validation and post-processing tools;
- an interactive dashboard to reduce the dependency on console commands.

1.4 Report organization

The report is organized into eight chapters.

Chapter 1 introduces the motivation of the project, the main objectives and the principal contributions of the work. It explains why Parkinson’s disease is considered a suitable case study for a multi-agent simulation and presents the general idea of combining distributed simulation with causal and multilevel graph analysis.

Chapter 2 provides the biological background required to understand the model. It briefly introduces the role of the substantia nigra, alpha-synuclein misfolding and aggregation, mitochondrial dysfunction, neuroinflammation and the involvement of glial cells. The chapter does not aim to provide a complete medical description of Parkinson’s disease, but rather to identify the biological mechanisms that are abstracted in the computational model.

Chapter 3 presents the theoretical background of the project. It introduces adaptive agents, multi-agent systems and the formal agent model used in the simulation. It also introduces multilevel graphs and explains why graph contraction is useful for moving from detailed event traces to more synthetic representations of system behavior.

Chapter 4 describes the simulation model. It is the central technical chapter of the report. It defines the agent categories used in the model, namely neurons, microglia, astrocytes, mitochondria, lysosomes, alpha-synuclein proteins and alpha-synuclein aggregates. For each agent type, the chapter describes its internal states, perceptions, actions, transition rules and effects on the environment. The chapter also discusses the parameter system, the stochastic components of the model, the simulation loop and the distributed execution through MPI.

Chapter 5 describes the post-run data pipeline. It explains how the raw simulation outputs are transformed after execution. In particular, it distinguishes the canonical semantic event log from the graph projections derived from it. It also describes the table-oriented export step, the event types used by the log and the role of spatial snapshots and tick metrics in the analysis workflow.

Chapter 6 presents the multilevel graph analysis. It describes how the project builds the graph hierarchy from simulation logs, starting from the detailed temporal causal graph and progressively applying time contraction, agent/state clustering and topological strongly connected component contraction. The chapter explains the interpretation of each graph level, from G0 to G3, and clarifies how each level supports a different type of analysis.

Chapter 7 describes the interactive dashboard used to configure, run and inspect the simulation. It presents the main views for parameter editing, simulation monitoring, post-run metrics, spatial reconstruction, initialization inspection, and multilevel graph exploration. The dashboard acts as the user-facing layer of the framework, connecting the model implementation with the analysis and visualization pipeline.

Chapter 8 concludes the report. It summarizes the main results of the project, discusses the limitations of the current model and proposes possible future extensions. These include improve-

ments to the biological modelling, experimentation over larger parameter spaces, further validation strategies and extensions of the visualization and analysis tools.

Overall, the report follows the complete workflow of the project: it starts from the biological and theoretical motivations, then presents the simulation model, explains how data are recorded and processed, and finally discusses how multilevel graph analysis is used to inspect the behavior produced by the simulation.

Chapter 2

Biological background

Parkinson's disease is the second most common neuro-degenerative disorder following Alzheimer's disease. The disease is classically characterized by overt motor symptoms which include slowness of movement, muscle rigidity and resting tremors. Furthermore, patients often experience non-motor symptoms that may manifest prior to the onset of motor problems.

This chapter introduces the main mechanisms behind the emergence of P.D., as described in the "Understanding Parkinson's Disease" video by Nature Video. [4]

Substantia Nigra

While Parkinson's disease affects various regions of the nervous system and multiple types of neurons, a primary focal point is the *substantia nigra pars compacta*, located in the midbrain. This region is a crucial component of a major neural pathway responsible for facilitating movement. The progressive death of dopaminergic neurons within the substantia nigra leads to the malfunction of this pathway, strongly contributing to the motor deficits observed in patients.

Protein Misfolding

A distinctive pathological mark found in the majority of Parkinson's cases is the presence of intracellular clumps of misfolded proteins, with the most common type being Lewy bodies. A key structural component of these aggregates is a misfolded protein known as alpha-synuclein. These misfolded proteins are capable of forming small repeated units called oligomers or developing into longer fibrous structures. Scientific evidences suggest that these forms of alpha-synuclein are highly toxic to neurons, overwhelming the cellular system responsible for protein-degradation and leading to neuronal death.

Mitochondrial dysfunction

Mitochondria are the organelles responsible for providing cells with the energy necessary for vital functions, they are also deeply implicated in the pathogenesis of Parkinson's disease, as they are responsible for critical processes that become impaired in Parkinson's disease. Furthermore, the vital recycling process designed to remove and replace old or damaged mitochondria is thought to be severely disrupted, resulting in toxic accumulation of dysfunctional mitochondria that can no longer sustain neuronal health.

Neuroinflammation and Glial Cells

The role of glial cells is another crucial aspect of Parkinson's pathology. As dopaminergic

neurons die off, specialized glial cells known as microglia are believed to clear the resulting cellular debris. This process triggers an immune response. Once activated, microglia release inflammatory cytokines that activate neighboring microglia, as well as astrocytes. The environment created by these activated cells promotes further neuroinflammation and cellular damage.

Disease spread and future therapeutic targets

As the disease progresses over time, pathology develops across more areas of the nervous system. An emerging hypothesis is that this widespread progression occurs through the direct transmission of misfolded alpha-synuclein between cells. Current active areas of research include developing methods to enhance the clearance of abnormal proteins, blocking cell-to-cell transmission of misfolded proteins, improving mitochondrial function, and targeting the underlying neuroinflammation.

Chapter 3

Theoretical background

3.1 Adaptive Agent Systems

An agent is an autonomous computational entity situated in a nenviroment and capable of acting to achieve its objectives. It perceives changes in the environment through sensors and influences the environment through actions performed by its actuators. Unlike a conventional program with completely predetermined behaviour, an agent controls its internal state and selects its actions according to the information available during execution.

3.1.1 Formality

Formally, an agent-based system can be represented as a pair (Ag, Env) , where Ag is the agent and Env is the environment. The interaction between them produces a *run*, a sequence of alternating environment states and agent actions.

$$r = e_0 \xrightarrow{\alpha_0} e_1 \xrightarrow{\alpha_1} \dots \xrightarrow{\alpha_{n-1}} e_n$$

The agent is modelled as a function

$$Ag : R^E \rightarrow Ac$$

where R^E is the set of runs ending in an environment state and Ac is the set of available actions. The environment is described by a state-transition function (τ) that determines the possible states resulting from the agents' actions.

An agent can be *reactive* when it responds promptly to environmental changes. On another hand, an agent is *adaptive* if it modifies its response according to both observed change and its internal state. Finally, a *proactive* (or *self-adaptive*) agent anticipates future conditions and acts in pursuit of longer-term objectives.

For the sake of the biological simulation, the adaptive agent seems to be the most appropriate, as it allows to link the current state of the agent, the global state of the environment and the final selection of the action.

An adaptive agent is defined as a 9-tuple $\langle I, E, Per, Ac, i_0, see, next, action, do \rangle$, where:

I is a set of possible internal states;

E is a set of environmental states;

Per is a set of possible perception for the agent;

Ac is a set of possible actions for the agent;

$i_0 \in I$ is the initial state;

$see : E \rightarrow Per$ is the function that generates an agent perception based on the current environment state;

$next : I \times Per \rightarrow I$ is the function that allows the internal states transitions;

$action : I \times Per \rightarrow Ac$ is the action mapping function, given the internal state and the perception, it decides an agent action;

$do : Ac \times E \rightarrow E$ actuates the action selected by *action*.

An adaptive agent system may contain several autonomous agents operating in the same dynamic environment. Such a system is called a *multiagent system*. Agents may have different goals, while the resulting state depends on their combined actions. Consequently, they must perceive one another and may need to communicate, cooperate, compete or coordinate access to shared resources. System-level behavior emerges from these local interactions and is not necessarily determined by any single agent. Adaptive agent systems are therefore suitable computational models for complex systems in which autonomous entities continuously adjust their behaviour in response to environmental changes and the actions of other agents.

One way in which agents can achieve such coordination is through *stigmergy*, a form of indirect communication mediated by the environment. In a stigmergic multiagent system, agents modify their shared environment by leaving traces which influence the subsequent actions of other agents. Inspired by the collective behaviour of social insects, this mechanism enables decentralized, adaptive and robust coordination without requiring explicit communication or centralized control.

3.2 Multilevel graph

Complex systems are often characterised by structures that can be observed at different scales. A detailed representation may be necessary to describe the behaviour of individual entities, whereas a more abstract representation may reveal global patterns that are difficult to identify at the original level. A *multilevel graph* provides a unified framework for representing these different degrees of abstraction within the same hierarchical structure.

3.2.1 Multilevel Graph Structure

Let $(G_0 = (V_0, E_0))$ be the graph representing the system at its finest level of granularity. Its vertices generally correspond to the elementary entities of the model, while its edges describe the relations among them. Starting from G_0 , progressively coarser graphs can be obtained by applying a sequence of contraction functions:

$$G_{i+1} = f_{c_i}(G_i)$$

where f_{c_i} identifies a collection of vertices or subgraphs of G_i and replaces each of them with a single *supernode*. Relations involving the contracted vertices are correspondingly mapped to edges between supernodes.

A multilevel graph can therefore be defined as a sequence

$$\mathcal{M} = \langle G_0, G_1, \dots, G_n \rangle$$

where each graph G_{i+1} is a contraction of the preceding graph G_i . Equivalently, the structure may be represented by the pair

$$\mathcal{M} = (G_0, \Gamma)$$

where G_0 is the root graph and $\Gamma = \langle f_{c_0}, f_{c_1}, \dots, f_{c_{n-1}} \rangle$ is the ordered sequence of contraction functions used to construct the hierarchy.

The root graph has the finest granularity, since its vertices represent the most elementary available entities. At higher levels, groups of related vertices are progressively aggregated into more complex units. As a result, the number of vertices and edges generally decreases, while the semantic meaning associated with each supernode becomes broader.

An important property of the model is the preservation of the correspondence between levels. A supernode is associated with the vertices or lower-level supernodes from which it originates. This relation makes it possible to move from an abstract representation to a more detailed one through a decontraction operation. The structure consequently supports both a bottom-up process, in which elementary entities are combined into increasingly abstract groups, and a top-down process, in which an aggregate can be inspected by recovering its internal composition.

3.2.2 Multilevel Analysis

Multilevel analysis consists in examining a system not only through some original graph, but also through the sequence of representations generated by the contraction process. Instead of computing a graph property at a single fixed scale, the same property can be measured at several levels of the hierarchy. Given a metric (m), the analysis may therefore consider the sequence

$$m(G_0), m(G_1), \dots, m(G_n)$$

This sequence describes how the property changes as the representation becomes progressively coarser. Measures such as graph size, density, connectivity, path length, clustering or degree distribution may exhibit different behaviours across the hierarchy. Some characteristics may be visible only at the finest level, whereas others may emerge after local details have been aggregated. The trajectory of a measure across the levels can itself be informative, since it describes the response of the system's topology to the abstraction process.

The approach combines a *local* and a *global* perspective. Lower levels preserve detailed relations among individual entities and are therefore suitable for analysing local configurations. Higher levels emphasize the relations among groups, communities or functional units, providing a more compact representation of the overall organisation. Intermediate levels connect these perspectives and may reveal structures that are hidden both in the fully detailed graph and in an excessively coarse representation.

As we will see in more details in later sections, this project uses the lower levels as the main source of truth (as they are based on the factual simulation data) and uses the higher levels for analysis. Higher-level graphs provide compact abstractions that can suggest global patterns. However, conclusions drawn at coarse levels should be validated by inspecting the corresponding lower-level structures through decontraction or by checking the original event data.

Chapter 4

Simulation

This chapter describes the computational model developed to represent a simplified set of mechanisms associated with the progression of Parkinson’s disease. The model is implemented in Python as a distributed multi-agent simulation using Repast4Py and the Message Passing Interface (MPI).

The simulated system is organized into two spatial scales. The first is an extracellular environment representing a simplified portion of the substantia nigra. It contains neurons, microglia and astrocytes. It may also contain extracellular α -synuclein and protein aggregates. The second scale consists of multiple intracellular environments, each associated with a specific neuron and containing mitochondria, lysosomes and intracellular α -synuclein.

This separation makes it possible to represent processes occurring at different biological scales while preserving the relationship between intracellular alterations, neuronal state transitions and modifications of the surrounding tissue.

The model follows the adaptive-agent formalism introduced in Section 3.1. At each simulation step, agents perceive the relevant portion of their environment, update their internal state, select an action and apply its effects. The repeated interaction of these entities determine the global evolution of quantities such as neuronal viability, dopamine production, inflammation, cellular debris and α -synuclein aggregation.

4.1 Agent Specification

The simulation includes seven main categories of entities: *neurons*, *microglia*, *astrocytes*, *mitochondria*, *lysosomes*, *alpha-synuclein molecules* and *protein aggregates*.

Each agent is characterized by a unique identifier, a type, an internal state, a spatial position and the habitat it lives in.

Although these entities are inspired by biological components, their behaviour is intentionally simplified. The objective is not to reproduce all known molecular pathways, but to represent a limited set of interactions that are relevant to the emergence of neurodegeneration within the model.

4.1.1 Ag_{neuron} : Dopaminergic Neuron Agent

The dopaminergic neuron is modelled as a hierarchical macro-agent. At the extracellular level, it perceives the inflammatory and toxic conditions of the substantia nigra. At the intracellular level, it contains a local habitat Env_{intra} in which mitochondria, lysosomes, alpha-synuclein molecules and protein aggregates interact.

The intracellular habitat does not directly determine a discrete ”*alive to dead*” condition. Instead, its state is summarized through continuous variables describing oxidative stress, intracellular debris, energy demand and pathological alpha-synuclein load. These variables, together with extracellular stress, determine the cumulative damage of the neuron.

Once the damage becomes sufficiently high, the neuron progresses from the healthy state to the compromised and apoptotic states. After a minimum apoptotic period and when the required

intracellular conditions are met, the neuron ruptures and releases debris and pathological alpha-synuclein into the extracellular environment.

Formal specification

The neuron is represented as

$$Ag_{neuron} = \langle I_N, E_N, Per_N, Act_N, i_{0N}, see_N, next_N, action_N, do_N \rangle$$

Its set of internal states is

$$I_N = \{\text{Healthy, Compromised, Apoptotic, Ruptured}\} \quad i_{0N} = \text{Healthy}$$

The available actions are

$$Act_N = \{\text{ReleaseDopamine, ReleaseAlpha, AbsorbAlpha, SignalStress, DumpDebris, Idle}\}$$

The neuron perception is represented by the tuple

$$Per_N = \langle pos, \alpha_{near}, inf_{ext}, deb_{ext}, ros, deb_{int}, deficit_{ATP}, \alpha_{load}, dam_{int}, dam_{cum} \rangle$$

where:

pos is the neuron's position in the extracellular environment;

α_{near} is the local density of extracellular pathological alpha-synuclein;

inf_{ext} is the extracellular inflammation level;

deb_{ext} is the amount of extracellular debris;

ros is the intracellular oxidative stress;

deb_{int} is the intracellular debris level;

$deficit_{ATP}$ is the unmet intracellular energy demand;

α_{load} is the intracellular pathological alpha-synuclein load;

dam_{int} is the computed intracellular damage;

dam_{cum} is the cumulative neuronal damage.

Damage computation

Alpha load The pathological alpha-synuclein load is computed from the molecular and aggregate content of the intracellular habitat:

$$\alpha_{load} = \text{clamp} \left(\frac{\sum_{a \in Env_{intra}} w_{\alpha}(a)}{W_{intra} H_{intra}} \right)$$

where $w_{\alpha}(a)$ is the pathological contribution of intracellular entity a , while W_{intra} and H_{intra} are the dimensions of the intracellular grid.

Intracellular damage The intracellular damage is defined as a weighted combination of oxidative stress, pathological protein load, and intracellular debris:

$$dam_{int} = \text{clamp}(w_{ros}ros + w_{\alpha}\alpha_{load} + w_{deb_{int}}deb_{int})$$

Extracellular stress Extracellular stress is computed as

$$dam_{ext} = \text{clamp}(w_{inf_{ext}} + w_{deb_{ext}} deb_{ext} + w_{\alpha_{near}} \alpha_{near})$$

Total stress The total stress perceived by the neuron is

$$\sigma_N = \text{clamp}\left(\frac{dam_{ext} + dam_{int}}{2}\right)$$

Cumulative cell damage The cumulative cell damage is updated according to

$$dam_{cum}(t+1) = \begin{cases} \text{clamp}(dam_{cum}(t) - r_{rec}) & \sigma_N \leq \theta_{low} \wedge i_{(t)N} \in \{\text{Healthy}, \text{Compromised}\} \\ \text{clamp}(dam_{cum}(t) + \min\{r_{acc}\sigma_N, \Delta_{max}\}) & \text{otherwise} \end{cases}$$

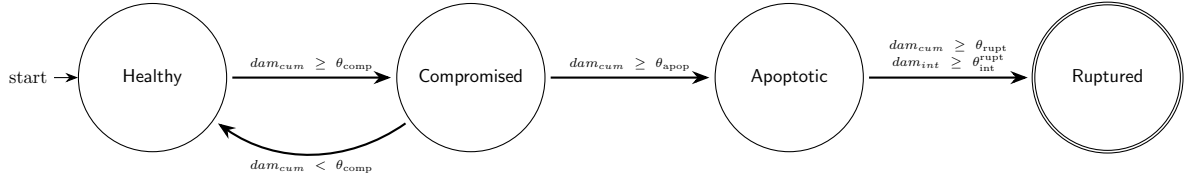
Here r_{rec} is the recovery rate, r_{acc} is the damage accumulation rate and Δ_{max} limits the maximum damage increase during one tick.

$next_N$ transition function

The cumulative damage first determines a candidate state:

$$\hat{s}_N = \begin{cases} \text{Ruptured} & \delta_{cell} \geq \theta_{rupt} \\ \text{Apoptotic} & \theta_{apop} \leq \delta_{cell} < \theta_{rupt} \\ \text{Compromised} & \theta_{comp} \leq \delta_{cell} < \theta_{apop} \\ \text{Healthy} & \delta_{cell} < \theta_{comp} \end{cases}$$

The ruptured state is absorbing. A compromised neuron may recover to the healthy state, whereas apoptosis is treated as irreversible in the current model.



$action_N$ mapping

$$action_N(i_{(t)N}, p_N) = \begin{cases} \text{DumpDebris} & i_{(t)N} = \text{Ruptured} \wedge \neg payload_{released} \\ \text{Idle} & i_{(t)N} = \text{Ruptured} \wedge payload_{released} \\ \text{AbsorbAlpha} & i_{(t)N} \neq \text{Apoptotic} \wedge \alpha_{near} \geq \theta_{near}^{high} \\ \text{ReleaseAlpha} & i_{(t)N} = \text{Apoptotic} \vee \alpha_{load} \geq \theta_{release} \\ \text{SignalStress} & i_{(t)N} = \text{Healthy} \wedge (i_{ext} \geq \theta_{i_{ext}}^{high} \vee d_{ext} \geq \theta_{debris}^{high}) \\ \text{ReleaseDopamine, SignalStress} & i_{(t)N} \in \{\text{Healthy}, \text{Compromised}\} \\ & \text{otherwise} \end{cases}$$

The order of the cases is significant. For example, a non-apoptotic neuron exposed to a sufficiently high local concentration of pathological α -synuclein attempts absorption before evaluating dopamine release or stress signaling.

do_N effects

The selected action is applied to the extracellular or intracellular environment and has an effect as summarized in the following table.

Action	Effect
ReleaseDopamine	Increases extracellular dopamine by an amount scaled according to the current neuronal state.
SignalStress	Increases the extracellular inflammation level by the configured stress release rate.
AbsorbAlpha	Attempts to transfer one nearby extracellular pathological α -synuclein entity or aggregate into the intracellular habitat.
ReleaseAlpha	Transfers intracellular pathological α -synuclein into the extracellular environment. Release is gradual before rupture.
DumpDebris	Releases intracellular debris, a fixed rupture payload, and all visible intracellular pathological α -synuclein. The fixed rupture payload is emitted only once.
Idle	Produces no additional environmental effect after the rupture payload has already been released.

Dopamine output Dopamine output depends on the neuronal state:

$$D_N(s_N) = r_D f_D(s_N)$$

where r_D is the base dopamine release rate and f_D is a state-dependent factor. Healthy neurons produce the full configured output, compromised neurons produce a reduced amount, while apoptotic and ruptured neurons normally provide no dopamine contribution.

Code implementation

In the codebase, the states set is defined in `NeuronState`, the perception in `NeuronPerception`, the actions in `NeuronAction`, the perception rules in `Neuron.see()`, the state transitions in `Neuron.next()`, the action mapping in `Neuron.action()` and the do effects in `Neuron.do()`.

4.1.2 $Ag_{microglia}$: Microglial Agent

The microglial agent represents an immune cell operating in the extracellular environment of the simulated substantia nigra. Its main functions are to scan the surrounding space, remove extracellular debris and contribute to the inflammatory response.

Formal specification

The microglial agent is represented as

$$Ag_{microglia} = \langle I_M, E_M, Per_M, Act_M, i_{0M}, see_M, next_M, action_M, do_M \rangle$$

Its set of internal states is

$$I_M = \{\text{Resting, Clearing, Activated}\} \quad i_{0M} = \text{Resting}$$

The available actions are

$$Act_M = \{\text{Scan, ClearDebris, ReleaseInflammation}\}$$

The microglial perception is represented by the tuple

$$Per_M = \langle pos, deb_{ext}, inf_{ext}, \alpha_{near} \rangle$$

where:

pos is the position of the microglial agent in the extracellular environment;

deb_{ext} is the current global amount of extracellular debris;

inf_{ext} is the current global inflammation level;

α_{near} is the local density of pathological alpha-synuclein within the microglial perception radius.

Environmental pressure computation

The environmental values are converted into normalized pressure values between zero and one. For a generic environmental value, the pressure is computed as

$$pressure(value, \theta^{low}, \theta^{high}) = \text{clamp} \left(\frac{value - \theta^{low}}{\theta^{high} - \theta^{low}} \right)$$

where θ are thresholds. If the high threshold is lower than or equal to the low threshold, the pressure is treated as a binary value:

$$pressure(value, \theta^{low}, \theta^{high}) = \begin{cases} 1 & value \geq \theta^{high} \\ 0 & value < \theta^{low} \end{cases}$$

Debris pressure The debris pressure is computed from the extracellular debris level:

$$\Pi_{deb_{ext}} = pressure(deb_{ext}, \theta_{deb_{ext}}^{low}, \theta_{deb_{ext}}^{high})$$

Inflammation pressure The inflammation pressure is computed from the extracellular inflammation level:

$$\Pi_{inf_{ext}} = pressure(inf_{ext}, \theta_{inf_{ext}}^{low}, \theta_{inf_{ext}}^{high})$$

Alpha-synuclein pressure The alpha-synuclein pressure is computed from the local density of pathological alpha-synuclein:

$$\Pi_{\alpha} = pressure(\alpha_{near}, \theta_{\alpha_{near}}^{low}, \theta_{\alpha_{near}}^{high})$$

Activation pressure The activation pressure is the greatest pressure produced by inflammation or nearby pathological alpha-synuclein:

$$\Pi_{act} = \max\{\Pi_{infect}, \Pi_{\alpha}\}$$

Debris pressure promotes the clearing state, while inflammation pressure and alpha-synuclein pressure promote the activated state.

Probabilistic transition sampling

Each possible state transition is associated with a probability. The agent samples a random value between zero and one:

$$u \sim Uniform(0, 1)$$

A transition is accepted when

$$u < p_{transition}$$

The probabilities used by the microglial agent are:

$$p_{resting \rightarrow clearing} = p_{activated \rightarrow clearing} = r_{clearing} \cdot \Pi_{debt}$$

$$p_{resting \rightarrow activated} = p_{clearing \rightarrow activated} = r_{activation} \cdot \Pi_{act}$$

Where $r_{clearing}$ and $r_{activation}$ are parameterable rates.

$next_M$ transition function

The next state depends on the current state, the environmental pressures and the sampled transition probabilities.

Resting state A resting microglial agent first attempts to enter the clearing state. The transition to the activated state is evaluated only when the transition to clearing is not accepted.

$$next_M(Resting, Per_M) = \begin{cases} \text{Clearing} & \Pi_{debt} > 0 \wedge u < p_{resting \rightarrow clearing} \\ \text{Activated} & \Pi_{act} > 0 \wedge u < p_{resting \rightarrow activated} \\ \text{Resting} & \text{otherwise} \end{cases}$$

The order of the conditions is significant. When both debris and activation pressures are present, the agent gives priority to the transition toward the clearing state.

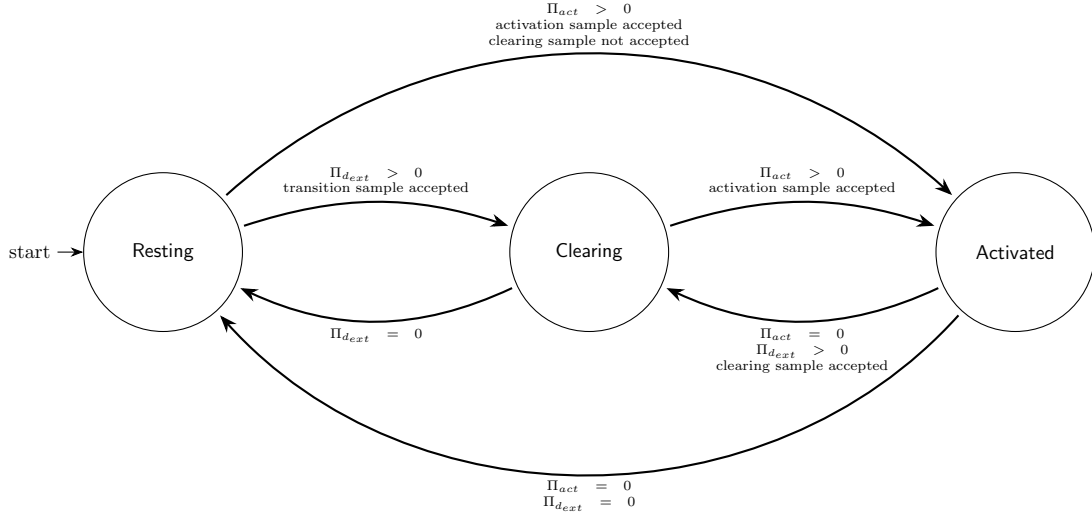
Clearing state A clearing microglial agent may become activated when inflammatory or pathological alpha-synuclein pressure is present. It may return to the resting state only when debris pressure is absent.

$$next_M(Clearing, Per_M) = \begin{cases} \text{Activated} & \Pi_{act} > 0 \wedge u < p_{clearing \rightarrow activated} \\ \text{Resting} & \Pi_{debt} = 0 \\ \text{Clearing} & \text{otherwise} \end{cases}$$

Activated state An activated microglial agent remains activated while inflammation or nearby pathological alpha-synuclein continues to produce activation pressure.

When activation pressure disappears, the agent may enter the clearing state if debris remains present. It may return to the resting state only when both activation pressure and debris pressure are absent.

$$next_M(Activated, Per_M) = \begin{cases} \text{Clearing} & \Pi_{act} = 0 \wedge \Pi_{debt} > 0 \wedge u < p_{activated \rightarrow clearing} \\ \text{Resting} & \Pi_{act} = 0 \wedge \Pi_{debt} = 0 \\ \text{Activated} & \text{otherwise} \end{cases}$$



action_M mapping

A resting microglial agent scans the environment. A clearing agent removes debris. An activated agent releases inflammation only when the environmental pressure is sufficiently high; otherwise, it scans the environment.

The pressure used to select the action of an activated agent is

$$\Pi_{action} = \max \left\{ \Pi_{\alpha}, \Pi_{infect}, \frac{\Pi_{debt}}{2} \right\}$$

The action mapping is

$$action_M(s_M, Per_M) = \begin{cases} \text{Scan} & s_M = \text{Resting} \\ \text{ClearDebris} & s_M = \text{Clearing} \\ \text{ReleaseInflammation} & s_M = \text{Activated} \wedge \Pi_{action} \geq \theta_{action} \\ \text{Scan} & s_M = \text{Activated} \wedge \Pi_{action} < \theta_{action} \end{cases}$$

An activated microglial agent does not necessarily release inflammation at every tick. It releases inflammation only when the combined environmental pressure reaches the configured action threshold.

do_M effects

The selected action is applied to the extracellular environment and has an effect as summarized in the following table.

Action	Effect
Scan	The agent may move to a randomly selected neighbouring position. Movement occurs according to the configured movement probability.
ClearDebris	Decreases the global extracellular debris level by the configured debris clearance rate.
ReleaseInflammation	Increases the global extracellular inflammation level by the configured inflammation release rate.

Scanning and movement During the scanning action, the agent samples whether it should move:

$$shouldMove = \begin{cases} \text{True} & u \leq p_{move} \\ \text{False} & u > p_{move} \end{cases}$$

When movement is accepted, the new position is selected randomly from the neighbouring positions of the current cell.

Debris clearance The clearing action modifies the extracellular debris level as follows:

$$deb_{ext}(t+1) = \max\{0, deb_{ext}(t) - r_{deb_{ext}clearance}\}$$

Inflammation release The inflammatory action modifies the extracellular inflammation level as follows:

$$inf_{ext}(t+1) = inf_{ext}(t) + r_{inf_{ext}release}$$

Code implementation

In the codebase, the state set is defined in `MicrogliaState`, the perception in `MicrogliaPerception`, the actions in `MicrogliaAction`, the perception rules in `Microglia.see()`, the state transitions in `Microglia.next()`, the action mapping in `Microglia.action()` and the environmental effects in `Microglia.do()`.

4.1.3 *Agastro*: Astrocyte Agent

The astrocyte agent represents an extracellular glial cell that regulates the inflammatory state of the simulated substantia nigra.

Its behaviour can be either protective or inflammatory. Under normal or moderately stressful conditions, the astrocyte provides support by reducing extracellular inflammation. When environmental stress persists over time, the agent may enter a reactive state and contribute to the inflammatory response.

The astrocyte perceives the global extracellular inflammation and debris levels. Since these quantities are shared by the whole simulated environment, a purely deterministic transition rule would cause all astrocytes to change state simultaneously. To avoid this synchronized behaviour, every astrocyte maintains an individual stress memory and independently samples its state transitions.

Formal specification

The Astrocyte agent is represented as

$$Ag_{astrocyte} = \langle I_{cyte}, E_{cyte}, Per_{cyte}, Act_{cyte}, i_{0cyte}, see_{cyte}, next_{cyte}, action_{cyte}, do_{cyte} \rangle$$

Its set of internal states is

$$I_{cyte} = \{\text{Supportive}, \text{Reactive}\} \quad i_{0cyte} = \text{Supportive}$$

The available actions are

$$Act_{cyte} = \{\text{Support}, \text{ReleaseInflammation}\}$$

The astrocyte perception is represented by the tuple

$$Per_{cyte} = \langle pos, inf_{ext}, deb_{ext} \rangle$$

where:

pos is the position of the astrocyte in the extracellular environment;

inf_{ext} is the current global extracellular inflammation level;

deb_{ext} is the current global amount of extracellular debris.

Environmental pressure computation

Inflammation pressure The inflammation pressure is computed as

$$\Pi_{infect} = pressure(infect, \theta_{infect}^{low}, \theta_{infect}^{high})$$

Debris pressure The debris pressure is computed as

$$\Pi_{debet} = pressure(\Pi_{debet}, \theta_{debet}^{low}, \theta_{debet}^{high})$$

Total stress pressure The astrocyte combines inflammation and debris into a single extracellular stress pressure:

$$\Pi_{tot} = \max \{ \Pi_{infect}, w_{debet} \cdot \Pi_{debet} \}$$

where w_{debet} is the configured weight assigned to extracellular debris.

The maximum operation means that inflammation directly determines stress when it is the dominant signal. Extracellular debris may also determine stress, but its contribution is scaled by w_{debet} .

Stress memory

The astrocyte does not react only to the instantaneous environmental pressure. Instead, it maintains an individual memory of the stress perceived over time.

Let $M_{stress}(t)$ be the stress memory at tick t . Its update is

$$M_{stress}(t+1) = \text{clamp}(d_{memory} \cdot M_{stress}(t) + (1 - d_{memory}) \cdot \Pi_{tot}(t))$$

where d_{memory} is the configured stress-memory decay factor.

The parameter d_{memory} controls how strongly previous conditions influence the current memory:

- when d_{memory} is close to zero, the stress memory follows the current environmental pressure rapidly;
- when d_{memory} is close to one, the stress memory changes slowly and preserves the effect of previous environmental conditions.

This mechanism allows persistent environmental stress to have a stronger effect than a brief isolated increase.

Probabilistic transition sampling

The probability of moving from the supportive state to the reactive state is

$$p_{supportive \rightarrow reactive} = \text{clamp}(r_{reactive} \cdot M_{stress})$$

where $r_{reactive}$ is the configured reactive transition rate.

The probability of moving from the reactive state to the supportive state is

$$p_{reactive \rightarrow supportive} = \text{clamp}(r_{recovery} \cdot (1 - M_{stress}))$$

where $r_{recovery}$ is the configured supportive recovery rate.

A high stress memory increases the probability of entering the reactive state and reduces the probability of returning to the supportive state.

$next_{cyte}$ transition function

The next state depends on the current astrocyte state, the stress memory and the sampled transition probability.

Supportive state A supportive astrocyte may become reactive according to the current stress memory:

$$next_{cyte}(\text{Supportive}, Per_{cyte}) = \begin{cases} \text{Reactive} & u < p_{supportive \rightarrow reactive} \\ \text{Supportive} & \text{otherwise} \end{cases}$$

Since $p_{supportive \rightarrow reactive}$ depends on M_{stress} , prolonged exposure to high inflammation or debris makes the reactive transition increasingly likely.

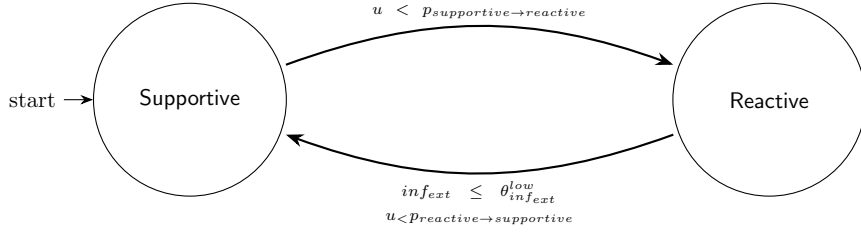
Reactive state A reactive astrocyte may recover only when the current inflammation level is below or equal to the configured low inflammation threshold.

$$next_{cyte}(\text{Reactive}, Per_{cyte}) = \begin{cases} \text{Supportive} & \text{if } inf_{ext} \leq \theta_{inflammation}^{low} \wedge u < p_{reactive \rightarrow supportive} \\ \text{Reactive} & \text{otherwise} \end{cases}$$

The recovery condition therefore contains two distinct requirements:

1. the current inflammation level must be sufficiently low;
2. the probabilistic recovery sample must be accepted.

Even when current inflammation decreases, a high stress memory may delay recovery because it reduces $p_{reactive \rightarrow supportive}$.



$action_{cyte}$ mapping

A supportive astrocyte always performs the support action. A reactive astrocyte does not automatically release inflammation. Its selected action depends on whether stress memory has crossed the configured inflammatory threshold.

$$action_{cyte}(i_{(t)cyte}, Per_{cyte}) = \begin{cases} \text{Support} & i_{(t)cyte} = \text{Supportive} \\ \text{ReleaseInflammation} & i_{(t)cyte} = \text{Reactive} \wedge M_{stress} \geq \theta_M \\ \text{Support} & i_{(t)cyte} = \text{Reactive} \wedge M_{stress} < \theta_M \end{cases}$$

This distinction separates the reactive state from the inflammatory action.

A recently activated astrocyte may remain reactive while continuing to reduce inflammation. It becomes inflammatory only after stress memory reaches θ_M .

do_{cyte} effects

The selected action modifies the extracellular inflammation field.

Action	Effect
Support	Reduces the global extracellular inflammation level by the configured support reduction rate.
ReleaseInflammation	Increases the global extracellular inflammation level. The released amount may be scaled according to the current stress memory.

Support action The support action reduces extracellular inflammation:

$$inf_{ext}(t+1) = \max\{0, inf_{ext}(t) - r_{support}\}$$

where $r_{support}$ is the configured inflammation reduction rate.

Inflammatory action The amount of inflammation released by a reactive astrocyte depends on the configured base release rate and on its stress memory.

The memory-dependent multiplier is

$$multiplier_{inf_{ext}} = (1 - w_M) + w_M \cdot M_{stress}$$

where w_M is the configured weight assigned to stress memory.

The released amount is

$$releasedInflammation = r_{inf_{ext}} \cdot multiplier_{inf_{ext}}$$

where $r_{inf_{ext}}$ is the base inflammation release rate.

The extracellular inflammation level is updated as

$$inf_{ext}(t+1) = inf_{ext}(t) + releasedInflammation$$

When $w_M = 0$, the released amount is equal to the base release rate and does not depend on stress memory.

When $w_M = 1$, the released amount is directly proportional to the current stress memory.

Intermediate values produce a weighted combination of the constant base output and the memory-dependent output.

Code implementation

In the codebase, the astrocytic states are defined in `AstrocyteState`, the perception structure in `AstrocytePerception` and the available actions in `AstrocyteAction`.

The perception rules are implemented in `Astrocyte.see()`, the stress-memory update and state-transition rules in `Astrocyte.next()`, the action mapping in `Astrocyte.action()` and the environmental effects in `Astrocyte.do()`.

4.1.4 *Agmito*: Mitochondrial Agent

The mitochondrial agent represents an intracellular organelle associated with a specific dopaminergic neuron.

The simulation does not model physical ATP production directly. Instead, the intracellular variable *ATPdemand* represents the amount of energetic need that is still unmet. A functional mitochondrion reduces this deficit, while a stressed or damaged mitochondrion contributes to oxidative stress and intracellular debris.

The mitochondrial lifecycle is influenced by four main local conditions:

- oxidative stress;
- unmet energy demand;
- nearby pathological alpha-synuclein aggregates;
- nearby intracellular debris.

Formal specification

The mitochondrial agent is represented as

$$Ag_{mitochondrion} = \langle I_{Mt}, E_{Mt}, Per_{Mt}, Act_{Mt}, i_{0Mt}, see_{Mt}, next_{Mt}, action_{Mt}, do_{Mt} \rangle$$

Its set of internal states is

$$I_{Mt} = \{\text{Healthy}, \text{Consumed}, \text{Damaged}, \text{Debris}\} \quad i_{0Mt} = \text{Healthy}$$

The available actions are

$$Act_{Mt} = \{\text{ReduceDemand}, \text{Stress}, \text{Fuse}, \text{Divide}\}$$

A mitochondrion might not decide to not perform an action, the absence of said action is represented by

$$action_{Mt} = \emptyset$$

This occurs when the mitochondrion has been assigned to a lysosome, when it is already debris or when no autonomous mitochondrial action should be performed.

The mitochondrial perception is represented by the tuple

$$Per_{Mt} = \langle pos, ros, ATP_{demand}, \alpha_{density}, deb_{density}, targetAssigned \rangle$$

where:

pos is the position of the mitochondrion in the intracellular habitat;

ros is the oxidative stress measured at the current intracellular position;

ATP_{demand} is the current unmet energetic need at the mitochondrial position;

$\alpha_{density}$ is the local density of alpha-synuclein aggregates within the configured perception radius;

$deb_{density}$ is the local intracellular debris density within the configured perception radius;

$targetAssigned$ is a boolean that indicates whether the mitochondrion is currently assigned to a lysosome as a degradation target.

Damage contexts

The mitochondrial agent uses threshold-based conditions to distinguish high damage, low damage and recovery contexts.

High-damage context The environment is considered highly damaging when at least one local damage signal reaches its high threshold:

$$highDamage = \begin{cases} \text{True} & ros \geq \theta_{os}^{high} \vee \alpha_{density} \geq \theta_{\alpha}^{high} \vee d_{density} \geq \theta_d^{high} \\ \text{False} & \text{otherwise} \end{cases}$$

Low-damage context The environment is considered low-damage only when all local damage signals are below or equal to their low thresholds:

$$lowDamage = \begin{cases} \text{True} & ros \leq \theta_{os}^{low} \wedge \alpha_{density} \leq \theta_{\alpha}^{low} \wedge d_{density} \leq \theta_d^{low} \\ \text{False} & \text{otherwise} \end{cases}$$

Recovery context A consumed mitochondrion may perform fusion-mediated recovery only when both energy demand and local damage are sufficiently low.

The low energy-demand threshold is

$$\theta_{ATP_{demand}}^{recovery} = \frac{\theta_{ATP_{demand}}^{high}}{2}$$

The recovery context is

$$recovery = \begin{cases} \text{True} & ATP_{demand} \leq \theta_{ATP_{demand}}^{recovery} \wedge lowDamage = \text{True} \\ \text{False} & \text{otherwise} \end{cases}$$

Toxicity and energy-demand pressure

Local toxicity Local toxicity is computed as a weighted combination of oxidative stress, aggregate density and debris density:

$$toxicity = \text{clamp}(0.4 \cdot ros + 0.4 \cdot \alpha_{density} + 0.2 \cdot deb_{density})$$

Energy-demand pressure The energy-demand pressure measures how far the current unmet energy demand is above the configured high-demand threshold:

$$\Pi_{ATP_{demand}} = \text{clamp} \left(\frac{ATP_{demand} - \theta_{ATP_{demand}}^{high}}{1 - \theta_{ATP_{demand}}^{high}} \right)$$

When energy demand is below the high-demand threshold, $\Pi_{ATP_{demand}} = 0$. The pressure increases toward one as unmet demand approaches its maximum value.

Healthy to consumed The probability that a healthy mitochondrion enters the consumed state is

$$p_{healthy \rightarrow consumed} = \text{clamp} (0.35 \cdot \Pi_{ATP_{demand}} + 0.65 \cdot toxicity)$$

Local toxicity has a greater influence than energy-demand pressure on the transition toward the consumed state.

Consumed to healthy A consumed mitochondrion may recover probabilistically only in a valid recovery context.

$$p_{consumed \rightarrow healthy} = \begin{cases} \text{clamp} (0.25 \cdot (1 - \Pi_{ATP_{demand}}) \cdot (1 - toxicity)) & recovery = \text{True} \\ 0 & recovery = \text{False} \end{cases}$$

The probability is highest when both toxicity and energy-demand pressure are low.

Consumed to damaged The probability that a consumed mitochondrion progresses to the damaged state is

$$p_{consumed \rightarrow damaged} = \text{clamp} (0.65 \cdot toxicity + 0.35 \cdot \Pi_{ATP_{demand}})$$

Damaged to consumed A damaged mitochondrion may partially recover to the consumed state only when all damage signals are low:

$$p_{damaged \rightarrow consumed} = \begin{cases} \text{clamp} (0.15 \cdot (1 - toxicity) \cdot (1 - \Pi_{ATP_{demand}})) & lowDamage = \text{True} \\ 0 & lowDamage = \text{False} \end{cases}$$

This transition represents partial recovery.

Irreversible damage The irreversible-damage pressure is

$$\Pi_{irreversible} = \text{clamp} (0.8 \cdot toxicity + 0.2 \cdot \Pi_{ATP_{demand}})$$

The probability of becoming debris is

$$p_{irreversible} = \begin{cases} \Pi_{irreversible} & \Pi_{irreversible} \geq \theta_{irreversible} \\ 0 & \Pi_{irreversible} < \theta_{irreversible} \end{cases}$$

The irreversible transition is therefore disabled until the combined pressure reaches the configured irreversible-damage threshold.

$$p_{consumed \rightarrow debris} = p_{damaged \rightarrow debris} = p_{irreversible}$$

$next_{Mt}$ transition function

The next mitochondrial state depends on its current state and the stochastic transition probabilities.

Healthy state A healthy mitochondrion may become consumed:

$$next_{Mt}(\text{Healthy}, Per_{Mt}) = \begin{cases} \text{Consumed} & u < p_{healthy \rightarrow consumed} \\ \text{Healthy} & \text{otherwise} \end{cases}$$

Consumed state A consumed mitochondrion may recover, progress to damage, become debris or remain consumed.

The possible transition probabilities are

$$\{p_{consumed \rightarrow healthy}, p_{consumed \rightarrow damaged}, p_{consumed \rightarrow debris}, p_{consumed \rightarrow consumed}\}$$

where

$$p_{consumed \rightarrow consumed} + p_{consumed \rightarrow healthy} + p_{consumed \rightarrow damaged} + p_{consumed \rightarrow debris} = 1$$

The resulting state is sampled from

$$next_{Mt}(\text{Consumed}, Per_{Mt}) \in \{\text{Healthy}, \text{Consumed}, \text{Damaged}, \text{Debris}\}$$

Damaged state A damaged mitochondrion may partially recover, become debris or remain damaged.

The possible transition probabilities are

$$\{p_{damaged \rightarrow consumed}, p_{damaged \rightarrow debris}, p_{damaged \rightarrow damaged}\}$$

where

$$p_{damaged \rightarrow damaged} + p_{damaged \rightarrow consumed} + p_{damaged \rightarrow Debris} = 1$$

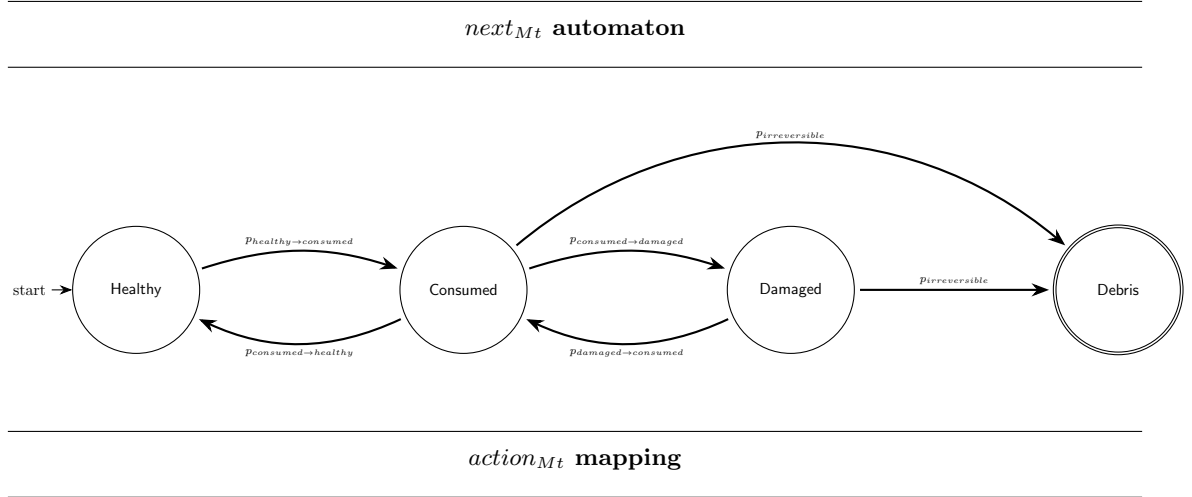
The resulting state is sampled from

$$next_{Mt}(\text{Damaged}, Per_{Mt}) \in \{\text{Consumed}, \text{Damaged}, \text{Debris}\}$$

Debris state The debris state is absorbing in the autonomous mitochondrial lifecycle:

$$next_{Mt}(\text{Debris}, Per_{Mt}) = \text{Debris}$$

A mitochondrion in the debris state does not recover autonomously. It can be restored only through successful lysosomal processing.



The selected action depends on the mitochondrial state, the perceived environment and whether the mitochondrion is currently assigned to a lysosome.

The action mapping is

$$action_{Mt}(i_{(t)Mt}, Per_{Mt}) = \begin{cases} \emptyset & targetAssigned = \text{True} \\ \emptyset & i_{(t)Mt} = \text{Debris} \\ \text{Divide} & i_{(t)Mt} = \text{Damaged} \\ \text{Fuse} & i_{(t)Mt} = \text{Consumed} \wedge recovery = \text{True} \\ \text{Stress} & i_{(t)Mt} = \text{Consumed} \wedge recovery = \text{False} \\ \text{Stress} & i_{(t)Mt} = \text{Healthy} \wedge highDamage = \text{True} \\ \text{ReduceDemand} & i_{(t)Mt} = \text{Healthy} \wedge highDamage = \text{False} \end{cases}$$

A mitochondrion assigned to a lysosome performs no autonomous action. This prevents the target from modifying the intracellular environment while it is being processed.

do_{Mt} effects

Before executing its selected action, every mitochondrion in the **Damaged** or **Debris** state is registered as a lysosomal degradation target.

The action effects are summarized in the following table.

Action	Effect
ReduceDemand	Reduces the neuron's unmet intracellular energy demand. The amount depends on mitochondrial state and on whether energy demand is above its high threshold.
Stress	Increases intracellular oxidative stress. A consumed mitochondrion also retains partial energetic functionality and reduces part of the unmet energy demand.
Fuse	Reduces oxidative stress and intracellular debris, then restores the consumed mitochondrion to the healthy state.
Divide	Represents dysfunctional mitochondrial fragmentation. It increases oxidative stress and intracellular debris and ensures that the mitochondrion is registered as a degradation target.
$\emptyset$	Produces no autonomous mitochondrial effect.

Energy-demand reduction

The amount by which a mitochondrion reduces unmet energy demand depends on its state. A multiplier is applied when current energy demand reaches or exceeds the high threshold:

$$multiplier_{ATP_{demand}} = \begin{cases} m_{ATP_{demand}} & ATP_{demand} \geq \theta_{ATP_{demand}}^{high} \\ 1 & ATP_{demand} < \theta_{ATP_{demand}}^{high} \end{cases}$$

where $m_{ATP_{demand}}$ is the configured high-demand reduction multiplier.

For a healthy mitochondrion,

$$demandReduction = r_{ATP_{demand}}^{healthy} \cdot multiplier_{ATP_{demand}}$$

For a consumed mitochondrion,

$$demandReduction = r_{ATP_{demand}}^{consumed} \cdot multiplier_{ATP_{demand}}$$

For damaged and debris mitochondria,

$$demandReduction = 0$$

The intracellular energy-demand field is updated as

$$ATP_{demand}(t+1) = ATP_{demand}(t) - demandReduction$$

Stress action

The stress action increases intracellular oxidative stress:

$$ros(t+1) = ros(t) + r_{stress}$$

where r_{stress} is the configured stress release rate.

When the mitochondrion is consumed, it still provides partial energetic support. Thus, the consumed state represents a stressed but partially functional mitochondrion.

Fusion action

The fusion action is available only to a consumed mitochondrion in a valid recovery context. It reduces oxidative stress:

$$ros(t+1) = ros(t) - r_{fusion}^{stress}$$

and reduces intracellular debris:

$$deb_{density}(t+1) = deb_{density}(t) - r_{fusion}^{debris}$$

where r_{fusion}^{stress} and r_{fusion}^{debris} are the configured fusion-related reduction rates. After these effects, the mitochondrial state is updated directly:

$$s_{(t+1)Mt} = \text{Healthy}$$

The fusion action therefore performs an action-driven repair in addition to the transitions sampled by the ordinary lifecycle automaton.

Division action

The division action is performed by a damaged mitochondrion. In the current model, it represents pathological fragmentation rather than healthy mitochondrial replication.

The action increases oxidative stress:

$$ros(t+1) = ros(t) + r_{damage}^{stress}$$

and increases intracellular debris:

$$deb_{density}(t+1) = deb_{density}(t) + r_{damage}^{debris}$$

where r_{damage}^{stress} is the damaged-mitochondrion stress release rate and r_{damage}^{debris} is the mitochondrial debris release rate.

The damaged mitochondrion is also registered in the lysosomal degradation buffer:

$$degradationTargets \leftarrow degradationTargets \cup \{Ag_{mitochondrion}\}$$

Lysosomal repair

A damaged or debris mitochondrion may be selected by a lysosome.

After successful lysosomal processing, the mitochondrion is restored directly to the healthy state:

$$repairByLysosome(\text{Damaged or Debris}) = \text{Healthy}$$

The pending mitochondrial action is cleared:

$$pendingAction = \emptyset$$

Lysosomal repair is external to the ordinary mitochondrial transition function. It may therefore restore both damaged and debris mitochondria even though the debris state is absorbing in the autonomous automaton.

Code implementation

In the codebase, the mitochondrial states are defined in `MitochondrionState`, the perception structure in `MitochondrionPerception` and the available actions in `MitochondrionAction`.

The perception rules are implemented in `Mitochondrion.see()`, the lifecycle transitions in `Mitochondrion.next()`, the action mapping in `Mitochondrion.action()` and the intracellular effects in `Mitochondrion.do()`.

4.1.5 $Ag_{lysosome}$: Lysosomal Agent

The lysosomal agent represents an intracellular clearance mechanism associated with a specific dopaminergic neuron. Its main role is to process damaged proteins, pathological alpha-synuclein aggregates and damaged mitochondria.

The lysosome does not independently search the whole intracellular environment for every degradable entity. Instead, damaged intracellular entities register as degradation targets in a shared buffer maintained by the owning neuron.

An active lysosome claims one available target and works on it for a number of simulation ticks. Once the required work duration has elapsed, the degradation attempt may have one of three mutually exclusive outcomes:

- the target is successfully degraded or repaired;
- the degradation attempt fails and the target is returned to the neuron's available-target buffer;
- the lysosome is overwhelmed by the target and becomes permanently inactive.

The duration and probability of degradation depend on the target type. Protein degradation and mitochondrial repair use configured constant durations, while aggregate degradation becomes more demanding as aggregate size increases.

A Lewy body always overwhelms the lysosome.

Formal specification

The lysosomal agent is represented as

$$Ag_{lysosome} = \langle I_L, E_L, Per_L, Act_L, i_{0L}, see_L, next_L, action_L, do_L \rangle$$

Its set of internal states is

$$I_L = \{\text{Inactive}, \text{Active}, \text{Overwhelmed}\} \quad i_{0L} = \text{Inactive}$$

The available actions are

$$Act_L = \{\text{Scan}, \text{SelectTarget}, \text{Degrade}, \text{Idle}\}$$

The lysosomal perception is represented by the tuple

$$Per_L = \langle pos, targets, task, \alpha_{density}, \Pi_{target} \rangle$$

where:

pos is the position of the lysosome in the intracellular environment of its owning neuron;

$targets$ is the collection of currently available and unassigned degradation targets;

$task$ is the target currently assigned to the lysosome, or $\emptyset$ when no target is assigned;

$\alpha_{density}$ is the local density of alpha-synuclein aggregates within the configured perception radius;

Π_{target} is the normalized amount of degradation work currently available in the intracellular environment.

The possible degradation targets are:

- damaged or pathological alpha-synuclein molecules;
- alpha-synuclein aggregates;
- damaged mitochondria;
- other intracellular entities explicitly registered as degradable.

Each lysosome belongs to one neuron and operates only inside that neuron's intracellular habitat.

Target pressure

The target pressure represents the proportion of intracellular entities that are currently available for degradation.

It is computed as

$$\Pi_{target} = \text{clamp} \left(\frac{|targets|}{totalIntracellularAgents - 1} \right)$$

where $|targets|$ is the number of currently available degradation targets.

The lysosome itself is excluded from the denominator. When the intracellular environment contains no other agent, the target pressure is zero.

A higher target pressure indicates that a larger portion of the intracellular environment requires lysosomal processing.

Probabilistic state-transition sampling

The transitions between the inactive and active states are probabilistic.

For each state update, the lysosome computes the probability of every available outgoing transition.

Inactive-to-active probability An inactive lysosome becomes more likely to activate when degradation targets or nearby aggregates are present.

The activation probability is

$$p_{inactive \rightarrow active} = \text{clamp} (\Pi_{target} + \alpha_{density} - \Pi_{target} \cdot \alpha_{density})$$

This combination is equivalent to the probability union of two independent signals.

It has the following properties:

- when both signals are zero, the activation probability is zero;
- when either signal reaches one, the activation probability reaches one;
- when both signals are positive, they jointly increase the activation probability without allowing it to exceed one.

Active-to-inactive probability An active lysosome may return to the inactive state when it has no assigned task and little or no degradation work is available.

The task pressure is defined as

$$\Pi_{task} = \begin{cases} 1 & task \neq \emptyset \\ 0 & task = \emptyset \end{cases}$$

The probability of returning to the inactive state is

$$p_{active \rightarrow inactive} = \text{clamp} ((1 - \Pi_{task}) \cdot (1 - \Pi_{target}) \cdot (1 - \alpha_{density}))$$

Consequently:

- an active lysosome cannot become inactive while it has an assigned task;
- the probability of becoming inactive increases when few targets are available;
- local aggregate presence reduces the probability of returning to the inactive state.

$next_L$ transition function

The ordinary state-transition function controls transitions between the inactive and active states.

The overwhelmed state is not reached through the ordinary activation automaton. It is entered as a consequence of an unsuccessful and damaging interaction with a degradation target.

Inactive state An inactive lysosome may become active according to the current activation probability:

$$next_L(\text{Inactive}, Per_L) = \begin{cases} \text{Active} & u < p_{inactive \rightarrow active} \\ \text{Inactive} & \text{otherwise} \end{cases}$$

Active state An active lysosome may return to the inactive state only when the sampled transition is accepted:

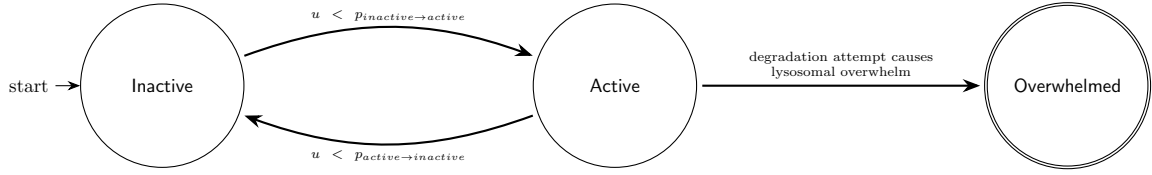
$$next_L(\text{Active}, Per_L) = \begin{cases} \text{Inactive} & u < p_{\text{active} \rightarrow \text{inactive}} \\ \text{Active} & \text{otherwise} \end{cases}$$

Since $p_{A \rightarrow I} = 0$ while a task is assigned, a lysosome remains active throughout an ongoing degradation process.

Overwhelmed state The overwhelmed state is absorbing:

$$next_L(\text{Overwhelmed}, Per_L) = \text{Overwhelmed}$$

An overwhelmed lysosome cannot recover in the current model.



$action_L$ mapping

The action selected by the lysosome depends on its current state and on whether a degradation target is already assigned.

The action mapping is

$$action_L(i_{(t)L}, Per_L) = \begin{cases} \text{Scan} & i_{(t)L} = \text{Inactive} \\ \text{SelectTarget} & i_{(t)L} = \text{Active} \wedge task = \emptyset \\ \text{Degrade} & i_{(t)L} = \text{Active} \wedge task \neq \emptyset \\ \text{Idle} & i_{(t)L} = \text{Overwhelmed} \end{cases}$$

do_L effects

The selected action is executed inside the intracellular environment of the owning neuron.

Action	Effect
Scan	Moves the lysosome to a randomly selected intracellular position within the configured movement radius.
SelectTarget	Selects one currently available degradation target and attempts to claim it from the neuron's shared target buffer.
Degrade	Advances the multi-tick degradation process for the currently assigned target. Once the required duration is completed, the degradation outcome is sampled.
Idle	Produces no effect. This action is performed permanently by an overwhelmed lysosome.

Scanning During the scanning action, the lysosome identifies all valid intracellular positions within its configured movement radius:

$$candidatePositions = neighbours(pos, moveRadius_L)$$

When at least one candidate position is available, the new position is selected uniformly:

$$pos(t+1) \sim Uniform(candidatePositions)$$

Unlike the microglial scanning action, lysosomal movement is not controlled by an additional movement probability. When scanning and at least one candidate position is available, the lysosome selects a new position.

Target selection An active lysosome with no assigned task reads the current list of available degradation targets.

When the list is non-empty, one target is selected randomly:

$$selectedTarget \sim Uniform(targets)$$

The lysosome then attempts to claim the target through the owning neuron's assignment mechanism. If the assignment succeeds,

$$task = selectedTarget$$

otherwise, the lysosome remains without a task.

The shared assignment mechanism prevents multiple lysosomes from processing the same target simultaneously.

Degradation duration

A degradation task may require several simulation ticks before its outcome can be resolved. The required duration depends on the target type.

Alpha-synuclein molecule For an individual alpha-synuclein molecule,

$$degradationTicks(target) = ticks_{protein}$$

where $ticks_{protein}$ is the configured protein-degradation duration.

Damaged mitochondrion For a damaged mitochondrion,

$$degradationTicks(target) = ticks_{mitochondrion}$$

where $ticks_{mitochondrion}$ is the configured mitochondrial-repair duration.

Alpha-synuclein aggregate For an aggregate,

$$degradationTicks(target) = ticks_{aggregate}^{base} + ticks_{aggregate}^{member} \cdot \max\{0, \alpha_{size} - 1\}$$

The base duration represents the minimum effort required to process an aggregate. The per-member component increases the required work as aggregate size grows.

Other targets For any other registered target,

$$degradationTicks(target) = 1$$

The lysosome stores the number of remaining work ticks in

$$ticksRemaining$$

At every degradation action:

$$ticksRemaining(t+1) = \begin{cases} ticksRemaining(t) - 1 & ticksRemaining(t) > 1 \\ 0 & ticksRemaining(t) \leq 1 \end{cases}$$

The degradation outcome is sampled only when $ticksRemaining$ reaches zero.

Degradation success probability

The probability of successful processing depends on the target type.

Alpha-synuclein molecule For an individual alpha-synuclein molecule,

$$p_{success} = p_{protein}^{base}$$

where $p_{protein}^{base}$ is the configured base degradation probability.

Damaged mitochondrion For a damaged mitochondrion,

$$p_{success} = p_{mitochondrion}^{repair}$$

where $p_{mitochondrion}^{repair}$ is the configured mitochondrial repair probability.

Alpha-synuclein aggregate For an aggregate,

$$p_{success}^{clamped} = \text{clamp}(p_{aggregate}^{base} + p_{aggregate}^{member} \cdot \alpha_{size})$$

Since overwhelm and degradation success are mutually exclusive outcomes, the effective success probability is limited by the remaining probability after overwhelm:

$$p_{success} = \min\{p_{success}^{clamped}, 1 - p_{overwhelm}\}$$

Other targets For any other target,

$$p_{success} = p_{protein}^{base}$$

Overwhelm probability

Only alpha-synuclein aggregates can overwhelm the lysosome in the current implementation.
For a non-aggregate target,

$$p_{overwhelm} = 0$$

For a non-Lewy-body aggregate,

$$p_{overwhelm} = \text{clamp}(p_{overwhelm}^{base} + p_{overwhelm}^{member} \cdot \alpha_{size})$$

Thus, larger aggregates may have a greater probability of overwhelming the lysosome.
For a Lewy body,

$$p_{overwhelm} = 1$$

Contact with a Lewy body therefore causes immediate and deterministic lysosomal overwhelm.

Degradation outcome

When the required work duration has elapsed, the lysosome samples

$$u_{deg} \sim \text{Uniform}(0, 1)$$

The degradation outcome is determined as follows:

$$\text{degradationOutcome} = \begin{cases} \text{Overwhelmed} & u_{deg} < p_{overwhelm} \\ \text{Success} & p_{overwhelm} \leq u_{deg} < p_{overwhelm} + p_{success} \\ \text{Failure} & \text{otherwise} \end{cases}$$

The three outcomes are mutually exclusive.

Successful degradation effects

The effect of successful degradation depends on the target type.

Target	Successful effect
Alpha-synuclein aggregate	The complete aggregate is removed from the intracellular habitat.
Damaged mitochondrion	The mitochondrion is repaired and removed from the neuron's degradation-target buffer.
Free alpha-synuclein	The molecule is marked as cleared and removed from the target buffer.
Alpha-synuclein belonging to an aggregate	The molecule is removed through the aggregate registry and unregistered as a degradation target.
Other target	The target is removed from the intracellular habitat.

After a successful outcome:

- the lysosomal task is cleared;
- the degradation work counter is reset;
- the lysosome remains active until the next state update.

Failed degradation effects

When degradation fails, the target is not removed or repaired.
Instead:

- the lysosomal assignment is cleared;
- the target is returned to the neuron's available-target buffer;
- the current degradation work is reset;
- the target may later be selected by the same or another lysosome.

The lysosome itself does not become overwhelmed after an ordinary failed attempt.

Overwhelm effects

When the target overwhelms the lysosome:

- the lysosomal state becomes **Overwhelmed**;
- the selected action becomes **Idle**;
- the current task is cleared;
- the work counter is reset;
- the target is returned to the neuron's available-target buffer;
- the lysosome can no longer process new targets.

The state update is

$$s_L(t + 1) = \text{Overwhelmed}$$

The overwhelmed state is permanent in the current model.

Code implementation

In the codebase, the lysosomal states are defined in `LysosomeState`, the perception structure in `LysosomePerception` and the available actions in `LysosomeAction`.

The perception rules are implemented in `Lysosome.see()`, the ordinary state-transition rules in `Lysosome.next()`, the action mapping in `Lysosome.action()` and the action effects in `Lysosome.do()`.

4.1.6 $Ag_{\alpha syn}$: Alpha-Synuclein Agent

The alpha-synuclein agent represents a single free alpha-synuclein protein. Its lifecycle is intentionally limited to the behaviour of individual proteins before they become part of an aggregate.

The agent can exist either inside a neuron, in an intracellular habitat, or in the extracellular environment. Intracellular proteins may misfold under oxidative and aggregate-seeded pressure. Once misfolded, they may express the intention to join an aggregate. The actual aggregation process is not performed by the alpha-synuclein agent itself. Instead, aggregation is resolved collectively by the aggregate registry, so that multiple candidate proteins can be handled consistently during the same simulation tick.

This separation prevents aggregation from depending on the sequential execution order of individual proteins.

Formal specification

The alpha-synuclein agent is represented as

$$Ag_{\alpha syn} = \langle I_{\alpha}, E_{\alpha}, Per_{\alpha}, Act_{\alpha}, i_{0\alpha}, see_{\alpha}, next_{\alpha}, action_{\alpha}, do_{\alpha} \rangle$$

Its set of internal states is

$$I_{\alpha} = \{\text{Monomer, Misfolded, Oligomer, LewyBody, Cleared}\} \quad i_{0\alpha} = \text{Monomer}$$

The available actions are

$$Act_{\alpha} = \{\text{Move, Stay}\}$$

The alpha-synuclein perception is represented by the tuple

$$Per_{\alpha} = \langle pos, ros, agg_{density}, neighbours \rangle$$

where:

pos is the current position of the protein in its habitat;

os is the intracellular oxidative stress perceived at the current position;

$agg_{density}$ is the local density of pathological alpha-synuclein aggregates within the configured perception radius;

$neighbours$ is the set of nearby agents perceived within the same radius.

The alpha-synuclein agent also stores its current compartment:

$$compartment \in \{\text{Intracellular, Extracellular}\}$$

An intracellular protein belongs to a specific owner neuron. An extracellular protein is no longer associated with an owner neuron.

Free protein condition

Only free proteins are updated as individual alpha-synuclein agents.

A protein is considered free when it is not already contained in an aggregate and it is not in a terminal state:

$$\alpha_{free} = \begin{cases} \text{True} & \alpha_{aggregateid} = \emptyset \wedge i_{(t)\alpha} \notin \{\text{Cleared, LewyBody}\} \\ \text{False} & \text{otherwise} \end{cases}$$

Only a free misfolded protein can become a candidate for oligomerization.

The condition for seeding oligomerization is

$$\alpha_{seed} = \begin{cases} \text{True} & \alpha_{free} = \text{True} \wedge i_{(t)\alpha} = \text{Misfolded} \wedge wantsOligomerization = \text{True} \\ \text{False} & \text{otherwise} \end{cases}$$

The flag *wantsOligomerization* is reset at every state update and consumed by the aggregate registry.

Aggregate load contribution

A free alpha-synuclein protein contributes to the local aggregate load according to its state.

$$\alpha_{weight}(s_\alpha) = \begin{cases} 0.25 & s_\alpha = \text{Misfolded} \\ 0.75 & s_\alpha = \text{Oligomer} \\ 1.00 & s_\alpha = \text{LewyBody} \\ 0.00 & \text{otherwise} \end{cases}$$

Once proteins are absorbed into an aggregate agent, they are removed from the active free-protein dynamics. Therefore, this weight is mainly relevant before absorption into an aggregate or when free protein placement is inspected directly.

Perception rules

Extracellular compartment An extracellular protein does not perceive intracellular oxidative stress or local intracellular aggregate density. Its perception is therefore

$$Per_\alpha = \langle pos, 0, 0, \emptyset \rangle$$

Intracellular compartment An intracellular protein perceives the local intracellular environment of its owner neuron.

Oxidative pressure Oxidative stress contributes to misfolding only when it reaches the configured high threshold.

The oxidative pressure is

$$\Pi_{ros} = \begin{cases} 0 & ros < \theta_{ros}^{high} \\ \text{clamp} \left(\frac{ros - \theta_{ros}^{high}}{1 - \theta_{ros}^{high}} \right) & ros \geq \theta_{ros}^{high} \end{cases}$$

A very small positive denominator is used internally when θ_{ros}^{high} is equal or extremely close to one.

Neighbourhood densities

The alpha-synuclein agent estimates two local densities from its neighbours.

Neighbour alpha density The neighbour alpha density is the fraction of perceived neighbouring agents that have the same alpha-synuclein type:

$$\alpha_{density} = \text{clamp} \left(\frac{\text{numberOfAlphaNeighbours}}{\text{numberOfNeighbours}} \right)$$

Neighbour aggregate density The neighbour aggregate density is computed from the aggregate-like weight of nearby alpha-synuclein proteins and aggregate agents:

$$agg_{density} = \text{clamp} \left(\frac{\sum_{a \in \text{neighbours}} w_{agg}(a)}{\text{numberOfNeighbours}} \right)$$

Misfolding probability

A monomer may misfold only when it is free and intracellular.

The probability of misfolding combines three components:

- a basal misfolding probability;
- an oxidative-stress component;
- an aggregate-seeded component.

The misfolding probability is

$$p_{misfolding} = \text{clamp}(p_{misfolding}^{basal} + w_{ros} \cdot \Pi_{ros} + w_{seeded} \cdot aggdensity)$$

where:

$p_{misfolding}^{basal}$ is the configured basal probability of misfolding;

w_{ros} is the configured weight of oxidative stress;

w_{seeded} is the configured weight of aggregate-seeded misfolding;

$aggregateDensity$ is the local aggregate density perceived in the intracellular habitat.

The basal component allows rare misfolding even in low-stress conditions, whereas oxidative stress and local aggregate density increase the probability of transition to the misfolded state.

Oligomerization probability

A misfolded protein does not immediately join an aggregate. It must first remain misfolded for at least a configured number of ticks.

Let $ticks_{misfolded}$ be the number of ticks already spent in the misfolded state.

$$eligibleForOligomerization = \begin{cases} \text{True} & ticks_{misfolded} \geq ticks_{misfolded}^{min} \\ \text{False} & ticks_{misfolded} < ticks_{misfolded}^{min} \end{cases}$$

If the protein is not yet eligible, the oligomerization probability is zero.

When eligibility is reached, the oligomerization probability is

$$p_{oligomerization} = \text{clamp}(scale_{oligomerization} \cdot (0.3 \cdot density_{\alpha} + 0.7 \cdot density_{aggregate}))$$

where $scale_{oligomerization}$ is the configured probability scale.

Neighbour aggregate density has a stronger contribution than neighbour alpha density. This reflects the model assumption that existing pathological structures more strongly promote further aggregation than free neighbouring proteins alone.

$next_{\alpha}$ transition function

The next state controls only the free-protein lifecycle. Aggregation itself is not performed inside $next_{\alpha}$. At the beginning of every state update,

$$wantsOligomerization = \text{False}$$

If the protein is not free, or if it is extracellular, its state does not change through this function:

$$next_{\alpha}(i_{(t)\alpha}, Per_{\alpha}) = i_{(t)\alpha} \quad \text{if } isFree = \text{False} \text{ or } compartment = \text{Extracellular}$$

Monomer state An intracellular free monomer may misfold according to the misfolding probability. The agent samples

$$u \sim \text{Uniform}(0, 1)$$

and applies

$$\text{next}_\alpha(\text{Monomer}, Per_\alpha) = \begin{cases} \text{Misfolded} & u < p_{\text{misfolding}} \\ \text{Monomer} & \text{otherwise} \end{cases}$$

When the transition to the misfolded state occurs, the counter $\text{ticks}_{\text{misfolded}}$ is reset:

$$\text{ticks}_{\text{misfolded}} = 0$$

Misfolded state A misfolded protein does not change its state directly inside the alpha agent. Instead, it may declare its intention to join an aggregate.

If the minimum misfolded residence time has not yet elapsed,

$$p_{\text{oligomerization}} = 0$$

Otherwise, the agent samples

$$u \sim \text{Uniform}(0, 1)$$

and updates the intention flag as

$$\text{wantsOligomerization} = \begin{cases} \text{True} & u < p_{\text{oligomerization}} \\ \text{False} & \text{otherwise} \end{cases}$$

The protein remains in the misfolded state until the aggregate registry absorbs it into an aggregate or until a lysosome clears it:

$$\text{next}_\alpha(\text{Misfolded}, Per_\alpha) = \text{Misfolded}$$

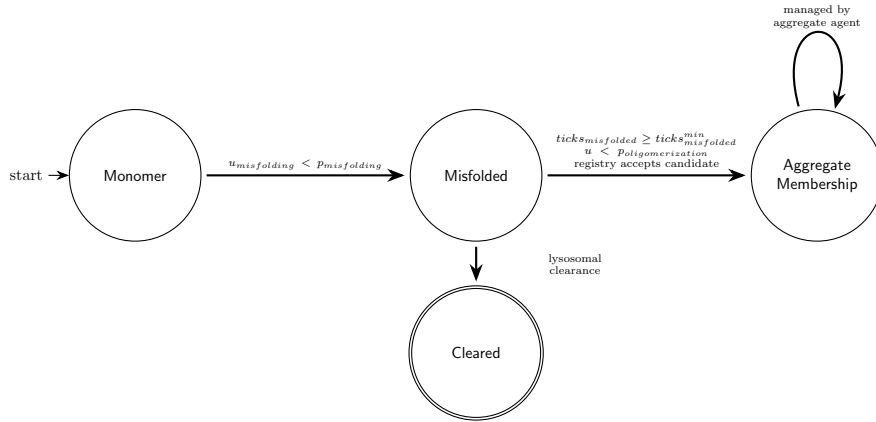
After the oligomerization attempt is evaluated, the misfolded-time counter is incremented:

$$\text{ticks}_{\text{misfolded}}(t + 1) = \text{ticks}_{\text{misfolded}}(t) + 1$$

Oligomer, Lewy body and cleared states The states **Oligomer** and **LewyBody** are assigned when the protein is incorporated into an aggregate by the aggregate registry.

The state **Cleared** is assigned when the protein is removed by lysosomal degradation.

These states are not reached through the ordinary free-protein transition function.



action_α mapping

The alpha-synuclein agent selects only movement-related actions.
The action mapping is

$$action_{\alpha}(i_{(t)\alpha}, Per_{\alpha}) = \begin{cases} \text{Stay} & isFree = \text{False} \\ \text{Stay} & compartment = \text{Extracellular} \\ \text{Move} & isFree = \text{True} \wedge compartment = \text{Intracellular} \end{cases}$$

Extracellular proteins are assigned the stay action because extracellular drift is handled by the model-level extracellular movement pass.

do_α effects

The selected action is applied inside the current habitat.

Action	Effect
Move	The free intracellular protein may move to a randomly selected nearby position according to the configured movement probability.
Stay	The protein does not perform autonomous movement. This action is used for extracellular proteins, cleared proteins and proteins already contained in aggregates.

Aggregation is intentionally absent from *do_α*. The alpha-synuclein agent can only express the intention to oligomerize; the collective registry decides whether and how aggregation occurs.

Movement A free intracellular protein first registers itself as degradable when it is misfolded:

$$registerAsDegradable = \begin{cases} \text{True} & compartment = \text{Intracellular} \wedge i_{(t)\alpha} = \text{Misfolded} \wedge isFree = \text{True} \\ \text{False} & \text{otherwise} \end{cases}$$

When this condition is true, the protein is added to the owner's lysosomal degradation-target buffer. After that, if the selected action is **Move**, the agent samples

$$u \sim Uniform(0, 1)$$

Movement occurs when

$$u_{move} \leq p_{move}$$

where p_{move} is the configured movement probability.
If movement is accepted, the candidate positions are

$$candidatePositions = neighbours(pos, moveRadius_{\alpha})$$

including the current position when allowed by the intracellular habitat.
The new position is selected uniformly:

$$pos(t + 1) \sim Uniform(candidatePositions)$$

If no valid candidate position is available, the protein remains in its current position.

Joining an aggregate

When the aggregate registry accepts a protein as part of an aggregate, the protein is no longer treated as an active free grid agent.

The operation is

$joinAggregate(aggregateId, s_{agg})$

and produces the following updates:

$aggregateId \leftarrow aggregateId_{selected}$

$i_{(t)\alpha} \leftarrow i_{(t)A}$

$pendingAction \leftarrow Stay$

$wantsOligomerization \leftarrow False$

$ticks_{misfolded} \leftarrow 0$

The resulting state may be **Oligomer** or **LewyBody**, depending on the aggregate structure assigned by the registry.

Lysosomal clearance

A free misfolded intracellular protein can be selected by a lysosome as a degradation target. When lysosomal clearance succeeds, the protein is marked as cleared:

$i_{(t)\alpha} \leftarrow Cleared$

$aggregateId \leftarrow \emptyset$

$pendingAction \leftarrow Stay$

$wantsOligomerization \leftarrow False$

$ticks_{misfolded} \leftarrow 0$

The cleared state is terminal for the free-protein lifecycle.

Release to extracellular environment

A protein may be released from a neuron into the extracellular environment, for example during neuronal rupture or pathological release.

The release operation updates the compartment and removes the owner-neuron association:

$compartment \leftarrow Extracellular$

$ownerNeuron \leftarrow \emptyset$

$pendingAction \leftarrow Stay$

$wantsOligomerization \leftarrow False$

After release, the protein no longer follows intracellular movement rules. Its extracellular motion is handled by the global extracellular drift mechanism.

Absorption into a neuron

An extracellular protein may be absorbed into a neuron.

The absorption operation updates the compartment and assigns a new owner neuron:

$compartment \leftarrow Intracellular$

$ownerNeuron \leftarrow neuron$

$pendingAction \leftarrow Stay$

wantsOligomerization $\leftarrow$ False

After absorption, the protein becomes part of the intracellular habitat of the new owner neuron and may again perceive oxidative stress and local aggregate density.

Code implementation

In the codebase, the alpha-synuclein states are defined in `AlphaSynucleinState`, the perception structure in `AlphaSynucleinPerception` and the available actions in `AlphaSynucleinAction`.

The compartment is represented by `AlphaSynucleinCompartment`, with the values `INTRACELLULAR` and `EXTRACELLULAR`.

The perception rules are implemented in `AlphaSynuclein.see()`, the free-protein lifecycle update in `AlphaSynuclein.next()`, the movement-action selection in `AlphaSynuclein.action()` and the movement or degradation-target registration effects in `AlphaSynuclein.do()`.

The operations that change the protein's structural role or compartment are:

- `AlphaSynuclein.join_aggregate();`
- `AlphaSynuclein.mark_cleared();`
- `AlphaSynuclein.release_to_environment();`
- `AlphaSynuclein.absorb_into_neuron();`

4.1.7 *Ag_A*: Alpha-Synuclein Aggregate Agent

The alpha-synuclein aggregate agent represents a pathological structure formed by multiple alpha-synuclein proteins.

Unlike the free alpha-synuclein agent, the aggregate does not autonomously decide how to grow. Aggregation is handled collectively by the `AggregateRegistry`. This registry processes all candidate proteins and aggregates located in the same intracellular grid cell, so that aggregation, merging and Lewy body maturation are resolved consistently.

The aggregate is therefore a simulation-visible unit representing a group of proteins. It contributes to local aggregate density, to the neuron's pathological alpha-synuclein load and to lysosomal degradation pressure.

Free alpha-synuclein proteins are removed from the active intracellular grid when they join an aggregate. From that point onward, the aggregate agent becomes the visible pathological structure.

Formal specification

The aggregate agent is represented as

$$Ag_{aggregate} = \langle I_A, E_A, Per_A, Act_A, i_A, see_A, next_A, action_A, do_A \rangle$$

Its set of internal states is

$$I_A = \{\text{Oligomer}, \text{LewyBody}\} \quad i_{0A} = \text{Oligomer}$$

The available actions are

$$Act_A = \{\text{Stay}\}$$

The aggregate perception is represented by the tuple

$$Per_A = \langle pos \rangle$$

where:

pos is the position of the aggregate in the intracellular habitat of its owning neuron.

The aggregate also stores the following structural information:

$$aggregateInfo = \langle aggregateId, members, size_A, ownerNeuron, wantsLewyBodyMaturation \rangle$$

where:

aggregateId is the identifier assigned by the aggregate registry;

members is the set of alpha-synuclein proteins represented by the aggregate;

size_A is the number of represented proteins;

ownerNeuron is the neuron whose intracellular habitat contains the aggregate;

wantsLewyBodyMaturation indicates whether the aggregate is eligible to act as a recruitment-competent pathological structure during the registry phase.

Aggregate size The size of the aggregate is the number of alpha-synuclein proteins represented by it:

$$size_A = |members|$$

An aggregate with no members is invalid and must be removed or rejected by the registry consistency checks.

Aggregate pathological weight The aggregate contributes to local pathological load through a state-dependent weight.

For an oligomer, the weight is

$$w_A = \min\{1.25, 0.75 + 0.05 \cdot \max\{0, size - 1\}\}$$

For a Lewy body, the weight is

$$w_A = \min\{2.00, 1.00 + 0.10 \cdot \max\{0, size - 1\}\}$$

Thus, larger aggregates contribute more strongly to pathological load, but the contribution is bounded. A Lewy body has a higher pathological weight than an oligomer of comparable size.

Recruitment competence The aggregate may recruit other pathological structures during the registry phase.

Recruitment competence is defined as

$$canRecruit = \begin{cases} \text{True} & s_{agg} = \text{LewyBody} \vee wantsLewyBodyMaturation = \text{True} \\ \text{False} & \text{otherwise} \end{cases}$$

A Lewy body is always recruitment-competent. An oligomer becomes recruitment-competent when the registry determines that it should mature into a Lewy body during the current aggregation pass.

next_A transition function

The ordinary transition function of the aggregate is simple.

The aggregate keeps its current state:

$$next_A(i_{(t)A}, Per_{agg}) = i_{(t)A}$$

The only internal flag updated by the ordinary transition step is

$$wantsLewyBodyMaturation = \begin{cases} \text{True} & i_{(t)A} = \text{LewyBody} \\ \text{False} & i_{(t)A} = \text{Oligomer} \end{cases}$$

This means that a Lewy body remains recruitment-competent between registry passes.

The transition from oligomer to Lewy body is not decided by *next_A*. It is a collective event resolved by the aggregate registry.

Lewy body maturation The aggregate registry may promote an oligomer to a Lewy body.
The maturation probability depends on aggregate size and on the configured Lewy body size threshold:

$$p_{maturation} = \text{clamp} \left(\frac{size_A}{\theta_{LewyBody}^{size}} \right)$$

where $\theta_{LewyBody}^{size}$ is the configured size threshold for Lewy body formation.
Maturation occurs when

$$u \sim \text{Uniform}(0, 1) < p_{maturation}$$

When maturation occurs, the aggregate state becomes

$$i_{(t)A} \leftarrow \text{LewyBody}$$

and every tracked member protein is mirrored to the Lewy body state:

$$i_{(t)A}(\text{member}) \leftarrow \text{LewyBody} \quad \text{for every } \text{member} \in \text{members}$$

Maturation is therefore a registry-level collective transition rather than an individual alpha-synuclein transition.

Aggregate creation A new aggregate may be created when at least two free misfolded alpha-synuclein proteins in the same intracellular cell want oligomerization.

The creation condition is

$$\text{canCreateAggregate} = \begin{cases} \text{True} & |\text{candidates}| \geq 2 \\ \text{False} & |\text{candidates}| < 2 \end{cases}$$

where

$$\text{candidates} = \{\alpha \mid \alpha.\text{canSeedOligomerization} = \text{True}\}$$

When an aggregate is created:

- a new aggregate identifier is assigned;
- the aggregate is added to the intracellular habitat at the cell position;
- the selected alpha-synuclein proteins are removed from the active grid;
- the selected proteins are registered as aggregate members;
- each member protein stores the aggregate identifier;
- each member protein is assigned the oligomer state;
- the aggregate is registered as a lysosomal degradation target.

After creation, the aggregate may immediately be tested for Lewy body maturation.

Adding a protein to an existing aggregate A free misfolded alpha-synuclein protein may be absorbed into an existing aggregate when it can seed oligomerization.

The absorption condition is

$$\text{canAddAlpha} = \begin{cases} \text{True} & \alpha.\text{canSeedOligomerization} = \text{True} \wedge \text{aggregateId is registered} \\ \text{False} & \text{otherwise} \end{cases}$$

When the protein is absorbed:

- the protein is removed from the intracellular grid;
- the protein is added to the aggregate member set;
- the aggregate size increases by one;
- the protein stores the aggregate identifier;
- the protein state becomes **Oligomer** if the aggregate is an oligomer;
- the protein state becomes **LewyBody** if the aggregate is a Lewy body;
- the protein no longer participates in free-protein movement or misfolding.

Aggregate merging When multiple recruitment-competent aggregates are located in the same intracellular cell, the registry merges them.

The target aggregate is selected as the largest aggregate in the cell:

$$targetAggregate = largest(aggregates)$$

For every other aggregate in the same merge group,

$$targetAggregate.members \leftarrow targetAggregate.members \cup sourceAggregate.members$$

The source aggregate is then removed from the registry and from the intracellular habitat.
After merging:

- the target aggregate contains all member proteins;
- the source aggregate is no longer active;
- the source aggregate is removed from lysosomal target tracking;
- the target aggregate is registered as a lysosomal degradation target;
- member states are synchronized with the target aggregate state.

If the target aggregate is a Lewy body, all members are mirrored to the Lewy body state.

Cell-level aggregation process Aggregation is processed cell by cell in the intracellular habitat.

For each intracellular grid cell, the registry identifies:

- free alpha-synuclein proteins that can seed oligomerization;
- existing aggregate agents located in the same cell;
- Lewy body aggregates;
- oligomers that are recruitment-competent.

The cell-level process follows an ordered priority.

Lewy body recruitment If one or more Lewy bodies are present in the cell, the largest Lewy body is selected as the seed.

Then:

- all other recruitment-competent aggregates in the cell are merged into the seed;
- all free oligomerization candidates in the cell are added to the seed;
- the process for that cell terminates.

This gives Lewy bodies priority over oligomer-based recruitment.

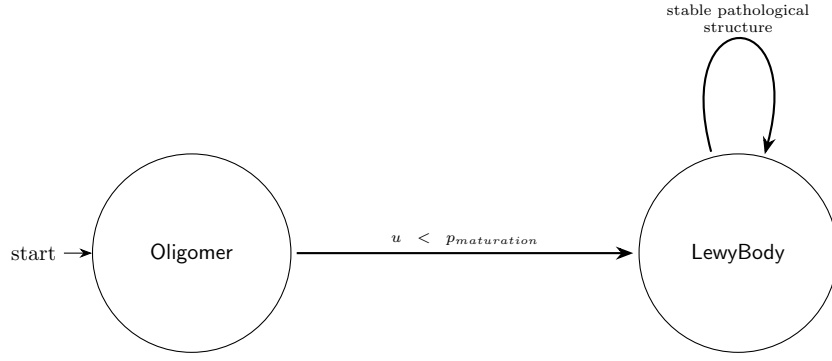
Oligomer recruitment If no Lewy body is present but at least one recruitment-competent oligomer is present, the largest recruitment-competent oligomer is selected as the seed.

Then:

- other recruitment-competent oligomers are merged into the seed;
- free oligomerization candidates are added to the seed;
- if the seed wants Lewy body maturation, it is promoted to **LewyBody**;
- the process for that cell terminates.

New aggregate formation If no recruitment-competent aggregate is present and at least two free oligomerization candidates are available, a new oligomeric aggregate is created.

After creation, the new aggregate is tested for Lewy body maturation.



action_A mapping

The aggregate does not currently move or choose an active behaviour.
 The action mapping is

$$action_A(i_{(t)A}, Per_A) = \text{Stay}$$

This holds for both oligomers and Lewy bodies.

do_A effects

The aggregate action does not modify the intracellular fields directly.
 The effect of *do_A* is to expose the aggregate as a possible lysosomal target inside the owning neuron.

$$do_A(\text{Stay}) = \begin{cases} registerAsDegradationTarget(Ag_{agg}) & ownerNeuron \neq \emptyset \\ \emptyset & ownerNeuron = \emptyset \end{cases}$$

Thus, the aggregate remains available for lysosomal processing as long as it is inside a neuron.

Lysosomal degradation

A lysosome may select an aggregate as a degradation target.

When aggregate degradation succeeds, the registry removes the aggregate and marks all tracked member proteins as cleared:

$$state(member) \leftarrow \text{Cleared} \quad \text{for every } member \in members$$

The aggregate is removed from:

- the aggregate registry;
- the intracellular habitat grid;
- the lysosomal degradation-target buffer;
- the member-tracking structure.

If only one member is removed from an aggregate, the registry updates the member set and clears that protein. If no members remain, the aggregate itself is removed.

Neuron transmission

Release to extracellular environment An aggregate may be released from a neuron into the extracellular environment, for example during neuronal rupture.

The release operation detaches the aggregate from its owner neuron:

$$ownerNeuron \leftarrow \emptyset$$

and sets the pending action to stay:

$$pendingAction \leftarrow \text{Stay}$$

When an aggregate is transferred outside a neuron, the registry may unregister it from the intracellular bookkeeping while preserving its member agents.

Absorption into a neuron An extracellular aggregate may be absorbed into a neuron.

The absorption operation assigns the new owner neuron:

$$ownerNeuron \leftarrow neuron$$

and sets the pending action to stay:

$$pendingAction \leftarrow \text{Stay}$$

When the aggregate is absorbed, it is registered in the target neuron's aggregate registry. Its member proteins are synchronized with the aggregate state:

$$state(member) = \begin{cases} \text{LewyBody} & state(Ag_{aggregate}) = \text{LewyBody} \\ \text{Oligomer} & state(Ag_{aggregate}) = \text{Oligomer} \end{cases}$$

The aggregate is then exposed again as a lysosomal degradation target in the new intracellular habitat.

Registry invariants

The aggregate registry enforces consistency between aggregates and their member proteins.

The main invariants are:

- every registered aggregate must contain at least one member;
- the aggregate member identifiers must match the tracked protein objects;
- every tracked member must point to the correct aggregate identifier;
- a cleared protein cannot remain tracked as an active aggregate member;
- a member protein absorbed into an aggregate must not remain active on the intracellular grid;
- a Lewy body aggregate must contain only Lewy body members;
- an aggregate cannot be active inside a habitat that does not own it.

These invariants ensure that aggregation remains a consistent collective process rather than a set of independent protein-level updates.

Code implementation

In the codebase, the aggregate states are defined in `AggregateState`, while the aggregate agent is implemented by `AlphaAggregate`.

The aggregate perception structure is `AggregatePerception` and the only aggregate action is represented by `AggregateAction.STAY`.

The perception rule is implemented in `AlphaAggregate.see()`, the ordinary state preservation in `AlphaAggregate.next()`, the action selection in `AlphaAggregate.action()` and the lysosomal-target exposure in `AlphaAggregate.do()`.

The methods `AlphaAggregate.add_member()`, `AlphaAggregate.add_members()` and `AlphaAggregate.mature_to_leywy_body()` update the aggregate structure.

The operations `AlphaAggregate.release_to_environment()` and `AlphaAggregate.absorb_into_neuron()` update the aggregate compartment ownership.

The collective aggregation logic is implemented by `AggregateRegistry`. In particular:

- `AggregateRegistry.process()` resolves the aggregation phase;
- `AggregateRegistry.create_aggregate()` creates a new oligomeric aggregate;
- `AggregateRegistry.add_alpha_to_aggregate()` absorbs a free misfolded protein into an aggregate;
- `AggregateRegistry.merge_aggregates()` merges two aggregate agents;
- `AggregateRegistry.mature_to_leywy_body()` promotes an oligomer to a Lewy body;
- `AggregateRegistry.validate_invariants()` checks consistency between aggregates, members and intracellular habitats;
- `AggregateRegistry.prune_cleared_aggregates()` removes aggregate structures whose tracked members have all been cleared.

4.2 Parameters and Stochasticity

The simulation is configured through a collection of YAML files. System-level settings are separated from parameters associated with individual agent types and biological mechanisms.

The main configuration categories include:

- simulation duration;
- random seed;
- spatial dimensions;
- initial agent population sizes;
- agent-specific transition probabilities;
- damage and recovery rates;
- α -synuclein production, clearance, and aggregation parameters;
- inflammatory response parameters;
- output directories;
- causal logging options;
- initialization logging options;
- temporal metric settings.

The separation of parameters from implementation code allows the same software model to be used for multiple experimental scenarios. For example, one experiment may reduce lysosomal efficiency, whereas another may increase the probability of mitochondrial damage or modify microglial activation.

4.2.1 Structural Parameters

Structural parameters determine the general organization of a simulation run. They include the final tick, the dimensions of the extracellular environment, the number of agents of each category and the number of MPI processes selected when starting the program.

These parameters affect both the interpretation of the simulated scenario and its computational cost. Increasing population size or spatial dimensions generally increases the number of interactions and the amount of data produced by the logging system.

4.2.2 Random Seed and Reproducibility

Since several transitions are stochastic, two executions with identical model parameters may produce different trajectories. The random seed is therefore part of the experimental configuration.

A fixed seed supports reproducibility during verification and debugging. Experiments intended to evaluate general behaviour should instead be repeated over multiple seeds.

Table 4.1: Main structural parameters of the simulation.

Parameter	Value	Description
Final simulation tick	500	Number of discrete time steps executed.
Grid width	10	Width of the extracellular spatial environment.
Grid height	10	Height of the extracellular spatial environment.
Initial neurons	21	Number of neurons created during initialization.
Initial microglia	15	Number of microglial agents.
Initial astrocytes	15	Number of astrocytic agents.

4.2.3 Interpretation of Parameters

The parameters used by the framework are computational abstractions. Unless they are explicitly calibrated against experimental observations, they must not be interpreted as estimated clinical or biological rates.

Their purpose is to control the qualitative behaviour of the model and to support comparative experiments. Consequently, conclusions should be expressed in terms of the model assumptions.

This section explains the effect of the parameters currently defined in `src/configuration/param`. The parameters are grouped according to the configuration file in which they appear.

Threshold Schema

Several perception thresholds are represented through a mean and a standard deviation:

```
threshold_name:
  mean: 0.5
  std: 0.05
```

The value `mean` represents the population average. The value `std` controls agent-to-agent heterogeneity. Each agent samples its resolved threshold once during configuration construction by using a Gaussian distribution:

$$resolvedThreshold \sim \text{clamp}(Gaussian(mean, std))$$

Higher mean values usually make agents less sensitive to the corresponding signal. Higher standard deviations reduce synchronized transitions across the population, but they may also create very sensitive or very insensitive outlier agents.

system.yaml

The file `system.yaml` contains global runtime, population, world and logging parameters.

Parameter	Current	Effect
<code>stop.at</code>	500	Number of scheduled ticks. Larger values expose slower or late-stage pathology.
<code>random.seed</code>	42	Global reproducibility seed for positions, sampled thresholds and stochastic events.
<code>external.population.neurons</code>	20	Global neuron count. MPI ranks partition this population, so the value describes the whole simulated system.
<code>external.population.microglia</code>	15	Global microglia count. More microglia increase debris clearance and eventual inflammatory release.
<code>external.population.astrocytes</code>	15	Global astrocyte count. More astrocytes increase supportive anti-inflammatory capacity and possible reactive inflammation.

Parameter	Current	Effect
<code>external.population.alpha</code>	0	Initial extracellular alpha-synuclein count. Values above zero seed extracellular pathology at startup.
<code>world.height</code>	10	Extracellular grid height.
<code>world.width</code>	10	Extracellular grid width.
<code>world.buffer_size</code>	2	Repast/MPI ghost-cell buffer.
<code>logging.enabled</code>	true	Default logging switch used when a sub-logger does not override it.
<code>logging.run_id</code>	default_run	Identifier written into output logs.
<code>logging.output_dir</code>	output	Root directory for runtime logs, rank logs, initialization logs and metrics.
<code>logging.causal.enabled</code>	true	Enables semantic event logging for G0 projection and mechanism analysis.
<code>logging.initialization.enabled</code>	true	Writes initial agent conditions and baseline alpha nodes.
<code>logging.scalar_stdout</code>	true	Prints main extracellular scalars every tick.
<code>logging.tick_metrics.csv</code>	true	Writes <code>tick_metrics.csv</code> with global per-tick scalars and compact state counts.
<code>logging.progress_stdout</code>	true	Prints progress and final summary from rank 0.
<code>logging.progress_interval</code>	25	Tick interval for progress messages.
<code>logging.summary_stdout</code>	true	Prints end-of-run counts for neurons, glia, alpha and aggregates.

substantia_nigra.yaml

The file `substantia_nigra.yaml` controls shared extracellular scalars and their population-level normalization.

Parameter	Current	Effect
<code>initial.debris</code>	0.1	Initial extracellular debris. Higher values activate debris-sensitive agents earlier.
<code>initial.inflammation</code>	0.08	Initial inflammation. Higher values push glia and neurons toward stress earlier.
<code>initial.dopamine</code>	0.8	Initial normalized dopamine before neuron release dynamics dominate.
<code>decay.debris</code>	0.03	Natural debris decay per tick. Higher values make debris self-resolve faster.
<code>decay.inflammation</code>	0.08	Natural inflammation decay per tick. Higher values make inflammatory plateaus harder to maintain.
<code>dopamine.smoothing</code>	0.35	Blend between previous dopamine and current normalized release. Higher values make dopamine more reactive and volatile.
<code>effects.baseline_debris_input</code>	0.003	Basal extracellular debris turnover per tick. Prevents a perfectly sterile zero-debris attractor.
<code>effects.debris_added_max_delta</code>	0.12	Maximum committed debris increase per tick after saturating population effects.
<code>effects.debris_removed_max_delta</code>	0.05	Maximum committed debris removal per tick. Higher values strengthen microglial cleanup.
<code>effects.debris_effect_scale</code>	0.8	Saturation scale for debris additions/removals. Higher values make raw population sums translate into larger scalar changes.
<code>effects.inflammation_added_max_delta</code>	0.08	Maximum committed inflammation increase per tick.

Parameter	Current	Effect
effects.inflammation_removed_max_delta	0.06	Maximum committed inflammation removal per tick.
effects.inflammation_effect_scale	0.8	Saturation scale for inflammatory additions/removals.

neuron.yaml

The file `neuron.yaml` controls extracellular perception, cumulative damage, alpha transmission and each neuron's intracellular habitat.

Parameter	Current	Effect
perception.per_radius	1	Radius for sensing extracellular alpha density.
th.nearby_alpha_high_threshold.mean	0.58	Mean nearby-alpha threshold for absorption/stress decisions. Higher values reduce alpha-driven response.
th.nearby_alpha_high_threshold.std	0.08	Heterogeneity in nearby-alpha sensitivity.
th.inflammation_high_threshold.mean	0.7	Mean inflammation threshold for stress signaling.
thresholds.inflammation_high_threshold.std	0.05	Heterogeneity in inflammatory sensitivity.
thresholds.debris_high_threshold.mean	0.6	Mean extracellular debris threshold for stress signaling.
thresholds.debris_high_threshold.std	0.05	Heterogeneity in debris sensitivity.
thresholds.alpha_load_release_threshold.mean	0.10	Mean intracellular alpha-load threshold for non-rupture alpha release. Higher values delay transmission.
thresholds.alpha_load_release_threshold.std	0.02	Heterogeneity in alpha-load release threshold.
thresholds.low_stress_threshold.mean	0.12	Total stress threshold below which cell damage can recover.
thresholds.low_stress_threshold.std	0.02	Heterogeneity in recovery tolerance.
thresholds.compromised_threshold	0.28	Cumulative damage required for Compromised .
thresholds.apoptotic_threshold	0.78	Cumulative damage required for Apoptotic . Higher values keep neurons Compromised longer.
thresholds.ruptured_threshold	0.985	Cumulative damage required for Ruptured .
damage.damage_accumulation_rate	0.22	Converts total stress into cumulative cell damage. Higher values accelerate pathology.
damage.damage_recovery_rate	0.022	Damage removed per low-stress tick for Healthy and Compromised neurons; it does not recover Apoptotic neurons.
damage.max_damage_increment_per_tick	0.05	Upper bound on damage gained in one tick, preventing rapid threshold skipping.
damage.min_ticks_compromised_before_apoptotic	25	Minimum ticks a neuron must remain Compromised before it can become Apoptotic .
damage.min_ticks_apoptotic_before_ruptured	20	Minimum ticks a neuron must remain Apoptotic before it can become Ruptured .
damage.apoptotic_internal_damage_threshold	0.35	Minimum internal damage required for a Compromised neuron to become Apoptotic ; blocks purely extracellular apoptotic jumps.
damage.rupture_internal_damage_threshold	0.45	Minimum internal damage required for rupture once cumulative damage is high enough.

Parameter	Current	Effect
damage.rupture_intracellular_debris_threshold	0.10	Minimum intracellular debris required for rupture once cumulative damage is high enough.
damage.inflammation_damage_weight	0.35	Inflammation contribution to extracellular neuron stress.
damage.debris_damage_weight	0.3	Extracellular debris contribution to neuron stress.
damage.alpha_damage_weight	0.31	Nearby alpha contribution to neuron stress. Lower values reduce alpha-driven rupture pressure.
rates.dopamine_release_rate	0.18	Dopamine released by a healthy neuron before normalization.
rates.dopamine_factor_healthy	1.0	State-specific dopamine factor for Healthy neurons.
rates.dopamine_factor_compromised	0.55	State-specific dopamine factor for Compromised neurons.
rates.dopamine_factor_apoptotic	0.15	Residual dopamine factor for Apoptotic neurons when they perform dopamine-coupled release.
rates.dopamine_factor_ruptured	0.0	Dopamine factor for Ruptured neurons.
rates.stress_inflammation_release_rate	0.04	Inflammation released by a neuron choosing the stress action.
rates.debris_release_rate	0.1	One-shot debris payload released by rupture.
alpha.alpha_absorption_rate	0.08	Probability that a candidate extracellular alpha/aggregate is absorbed.
alpha.alpha_release_amount	0.04	Fraction of eligible alpha pathology released by non-ruptured leakage.
alpha.alpha_release_dopamine_fraction	0.35	Fraction of state-dependent dopamine retained while a functional neuron releases alpha.
intracellular.grid.height	5	Internal grid height. Larger values dilute local intracellular encounters.
intracellular.grid.width	5	Internal grid width. Larger values dilute local intracellular encounters.
intracellular.population.alpha	40	Initial alpha-synuclein proteins per neuron. Higher values increase misfolding and aggregation substrate.
intracellular.population.mitochondria	10	Initial mitochondria per neuron.
intracellular.population.lysosomes	5	Initial lysosomes per neuron. Higher values increase cleanup and mitochondrial repair capacity.
intracellular.scalars.energy_demand_baseline	0.5	Baseline unmet energy demand.
intracellular.decay.energy_demand_recovery_rate	0.02	Pull of energy demand back toward baseline.
intracellular.decay.oxidative_stress_decay	0.005	Natural oxidative stress decay. Higher values reduce alpha misfolding pressure.
intracellular.decay.intracellular_debris_decay	0.005	Natural intracellular debris decay.
ic.damage.internal_damage.oxidative_weight	0.35	Oxidative stress contribution to internal neuron damage.
ic.damage.internal_damage_aggregate_weight	0.5	Alpha aggregate/load contribution to internal neuron damage.
ic.damage.internal_damage_debris_weight	0.15	Intracellular debris contribution to internal neuron damage.

alpha.yaml

The file `alpha.yaml` controls free alpha-synuclein proteins before they are absorbed into an aggregate.

Parameter	Current	Effect
perception.perception_radius	1	Radius for reading local oxidative stress, aggregate density and neighbors.
movement.move_radius	1	Maximum movement radius per tick.
movement.move_probability	0.35	Probability of moving when free. Higher values increase mixing and encounters.
rates.basal_misfold_probability	0.002	Baseline per-tick misfolding probability without stress. Higher values create widespread spontaneous pathology.
rates.aggregate_seeded_misfold_weight	0.10	Misfolding pressure from local aggregates. Higher values strengthen prion-like seeding.
rates.oxidative_misfolding_weight	0.45	Misfolding pressure from oxidative stress above threshold.
rates.oligomerization_probability_scale	0.25	Probability scale for misfolded alpha wanting oligomerization.
rates.min_misfolded_ticks_before_oligomerization	4	Minimum ticks spent misfolded before oligomerization can be attempted.
thresholds.oxidative_stress_high_threshold.mean	0.58	Mean oxidative stress threshold for stress-driven misfolding. Higher values reduce sensitivity.
thresholds.oxidative_stress_high_threshold.std	0.07	Heterogeneity in oxidative-stress sensitivity.

lysosome.yaml

The file `lysosome.yaml` controls lysosomal targeting, degradation, mitochondrial repair and overwhelm.

Parameter	Current	Effect
perception.perception_radius	1	Radius for reading local aggregate density and nearby targets.
movement.move_radius	1	Movement radius during scanning.
degradation.base_degradation_probability	0.5	Probability of clearing non-aggregate protein targets.
degradation.protein_degradation_ticks	2	Ticks required before resolving base protein degradation.
repair.mitochondrion_repair_ticks	3	Ticks required before resolving mitochondrial repair.
repair.mitochondrion_repair_probability	0.55	Probability that mitochondrial repair succeeds.
aggregate.degradation_ticks_base	1	Base ticks required before attempting aggregate degradation.
aggregate.degradation_ticks_per_member	1	Additional ticks per aggregate member.
aggregate.degradation_probability_base	0.18	Base probability of degrading a non-Lewy aggregate.
aggregate.degradation_probability_per_member	0.035	Degradation probability added per member.
aggregate.overwhelm_probability_base	0.11	Base probability that an aggregate overwhelms the lysosome.
aggregate.overwhelm_probability_per_member	0.06	Overwhelm probability added per aggregate member.

mitochondrion.yaml

The file `mitochondrion.yaml` controls mitochondrial lifecycle, oxidative stress, debris and energy-demand interaction.

Parameter	Current	Effect
perception.perception_radius	1	Radius for sensing local aggregate and debris density.
thresholds.energy_demand_high_threshold.mean	0.7	Mean threshold for high unmet energy demand.
thresholds.energy_demand_high_threshold.std	0.05	Heterogeneity in energy-demand sensitivity.
thresholds.oxidative_stress_high_threshold.mean	0.7	Mean high oxidative-stress threshold.
thresholds.oxidative_stress_high_threshold.std	0.05	Heterogeneity in stress vulnerability.
thresholds.oxidative_stress_low_threshold.mean	0.45	Mean low oxidative-stress threshold for recovery-oriented behavior.
thresholds.oxidative_stress_low_threshold.std	0.03	Heterogeneity in low-stress threshold.
thresholds.aggregate_density_high_threshold.mean	0.55	Mean high local aggregate-density threshold.
thresholds.aggregate_density_high_threshold.std	0.05	Heterogeneity in aggregate sensitivity.
thresholds.aggregate_density_low_threshold.mean	0.3	Mean low local aggregate-density threshold.
thresholds.aggregate_density_low_threshold.std	0.03	Heterogeneity in low aggregate-density threshold.
thresholds.debris_density_high_threshold.mean	0.55	Mean high local debris-density threshold.
thresholds.debris_density_high_threshold.std	0.05	Heterogeneity in debris sensitivity.
thresholds.debris_density_low_threshold.mean	0.3	Mean low local debris-density threshold.
thresholds.debris_density_low_threshold.std	0.03	Heterogeneity in low debris-density threshold.
thresholds.irreversible_damage_threshold.mean	0.65	Mean threshold for irreversible mitochondrial damage risk.
thresholds.irreversible_damage_threshold.std	0.03	Heterogeneity in irreversible damage vulnerability.
rates.stress_release_rate	0.1	Oxidative stress added by pathological mitochondrial behavior.
rates.damage_stress_release_rate	0.15	Oxidative stress added by damaged mitochondria.
rates.debris_release_rate	0.04	Intracellular debris added by damaged/debris mitochondria.
rates.fusion_stress_reduction_rate	0.02	Oxidative stress reduction from fusion/repair behavior.
rates.fusion_debris_reduction_rate	0.01	Intracellular debris reduction from fusion/repair behavior.
rates.healthy_energy_demand_reduction_rate	0.008	Energy-demand reduction from healthy mitochondria.
rates.consumed_energy_demand_reduction_rate	0.003	Energy-demand reduction from consumed mitochondria.
rates.high_demand_reduction_multiplier	1.5	Multiplier applied when energy demand is high.

microglia.yaml

The file microglia.yaml controls extracellular immune sensing, transitions and actions.

Parameter	Current	Effect
perception.per_radius	1	Radius for perceiving nearby extracellular alpha.
thresholds.debris_high_threshold.mean	0.6	Mean threshold for high debris. Lower values make clearing more likely.
thresholds.debris_high_threshold.std	0.05	Heterogeneity in high-debris sensitivity.
thresholds.debris_low_threshold.mean	0.2	Mean threshold below which debris pressure is low.
thresholds.debris_low_threshold.std	0.03	Heterogeneity in low-debris threshold.

Parameter	Current	Effect
thresholds.inflammation_high_threshold.mean	0.7	Mean threshold for inflammation-driven activation.
thresholds.inflammation_high_threshold.std	0.05	Heterogeneity in inflammatory sensitivity.
thresholds.inflammation_low_threshold.mean	0.3	Mean threshold below which inflammation pressure is low.
thresholds.inflammation_low_threshold.std	0.03	Heterogeneity in low-inflammation threshold.
thresholds.nearby_alpha_high_threshold.mean	0.5	Mean threshold for alpha-driven activation.
thresholds.nearby_alpha_high_threshold.std	0.05	Heterogeneity in alpha sensitivity.
thresholds.nearby_alpha_low_threshold.mean	0.2	Mean threshold below which nearby alpha pressure is low.
thresholds.nearby_alpha_low_threshold.std	0.03	Heterogeneity in low-alpha threshold.
rates.debris_clearance_rate	0.035	Raw debris removal contributed by a clearing microglia.
rates.inflammation_release_rate	0.024	Raw inflammation added by activated microglia when action pressure is high enough.
movement.move_probability	0.5	Probability of moving while scanning.
dynamics.activation_transition_rate	0.35	Scale for probabilistic transition to Activated .
dynamics.clearing_transition_rate	0.45	Scale for probabilistic transition to Clearing .
dynamics.recovery_transition_rate	0.25	Scale for recovery to Resting when pressure is low.
dynamics.inflammatory_action_threshold	0.28	Minimum contextual pressure required for activated microglia to release inflammation.

astrocyte.yaml

The file `astrocyte.yaml` controls Supportive/Reactive transitions and inflammation modulation.

Parameter	Current	Effect
thresholds.inflammation_high_threshold.mean	0.55	Mean threshold for becoming reactive due to inflammation.
thresholds.inflammation_high_threshold.std	0.1	Heterogeneity in inflammatory reactivity.
thresholds.inflammation_low_threshold.mean	0.25	Mean threshold below which inflammation is low for recovery logic.
thresholds.inflammation_low_threshold.std	0.05	Heterogeneity in low-inflammation threshold.
thresholds.debris_high_threshold.mean	0.55	Mean threshold for debris-driven reactivity.
thresholds.debris_high_threshold.std	0.1	Heterogeneity in debris sensitivity.
thresholds.debris_low_threshold.mean	0.15	Mean threshold below which debris is low.
thresholds.debris_low_threshold.std	0.04	Heterogeneity in low-debris threshold.
rates.support_inflammation_reduction_rate	0.014	Raw inflammation removal by supportive astrocytes.
rates.inflammation_release_rate	0.015	Raw inflammation added by reactive astrocytes when stress memory is high enough.
dynamics.stress_memory_decay	0.85	Persistence of astrocyte stress memory. Higher values make reactivity longer-lasting.
dynamics.reactive_transition_rate	0.45	Scale for probabilistic transition to Reactive .
dynamics.supportive_recovery_rate	0.16	Scale/probability of recovery to Supportive .

Parameter	Current	Effect
<code>dynamics.inflammatory_memory_threshold</code>	0.5	Stress-memory level required for a reactive astrocyte to release inflammation.
<code>dynamics.inflammation_memory_weight</code>	0.8	Inflammation contribution to astrocyte stress memory.
<code>dynamics.debris_stress_weight</code>	0.2	Debris contribution to astrocyte stress memory.

4.3 Simulation Loop

The simulation advances through discrete time steps, referred to as *ticks*. During each tick, agents observe their local conditions, update their state, and generate effects on the model.

Execution is coordinated through Repast4Py, while MPI is used to distribute agents and computational work among multiple processes.

Algorithm 1 provides a high-level description of the execution procedure.

Algorithm 1 Distributed simulation loop

- 1: Load system and agent parameters
 - 2: Initialize the MPI communicator
 - 3: Create the distributed context and spatial projections
 - 4: Generate extracellular and intracellular agents
 - 5: Assign initial positions and internal states
 - 6: Write initialization logs, if enabled
 - 7: **for** $t \leftarrow 0$ to $T - 1$ **do**
 - 8: Update agent perceptions
 - 9: Compute internal state transitions
 - 10: Select agent actions
 - 11: Apply local actions
 - 12: Resolve collective molecular processes
 - 13: Synchronize distributed agent state
 - 14: Compute rank-local metrics
 - 15: Reduce local metrics into global values
 - 16: Write temporal and causal logs
 - 17: Advance the scheduler
 - 18: **end for**
 - 19: Merge rank-local outputs
 - 20: Close output files
 - 21: Terminate execution
-

4.3.1 Initialization

The initialization phase constructs the distributed simulation state before the first tick is executed. The standard entry point of the project is the `simulate` command exposed by `main.py`, typically launched through MPI as follows:

```
mpiexec -n N python main.py simulate
```

where `N` is the number of MPI processes used for the execution. Each process corresponds to a different MPI rank. The same simulation can therefore be executed with one rank for a minimal local run, or with multiple ranks for a distributed execution.

At startup, the engine loads the system configuration and the agent-specific configuration files from the parameter directory. These files define the simulation duration, the random seed, the extracellular grid dimensions, the initial global populations and the parameters used by the different agent types.

The population values specified in the configuration are global values. For example, the configured number of neurons, microglia and astrocytes refers to the whole simulated substantia nigra, not to the population owned by a single rank. During initialization, these global populations are partitioned across the available MPI ranks. Each rank creates and manages only its local subset of agents.

The extracellular environment is represented as a distributed spatial structure. Each rank owns a portion of this structure and places its local extracellular agents inside it. The Repast4Py distributed context and spatial projection maintain the association between agents, positions and rank ownership.

Neuron initialization also creates a nested intracellular environment. Each neuron owns an internal habitat containing the initial alpha-synuclein proteins, mitochondria and lysosomes specified by the neuron configuration. These intracellular agents belong to the neuron that creates them and are therefore initialized together with their owner.

The random seed controls stochastic aspects such as initial positions, sampled thresholds and agent-specific heterogeneous parameters. In a distributed setting, reproducibility depends on both the configuration and the selected MPI setup. Consequently, the same configuration executed with the same number of ranks should produce the same initial distributed state.

At the end of initialization, every MPI rank has completed the creation of its local agents and intracellular structures. The distributed context contains the global simulation partitioned across ranks, and the scheduler can start the tick-based execution.

4.3.2 Distributed Execution

After initialization, the simulation advances through discrete ticks. During each tick, every MPI rank updates the agents that it owns. Most computations are local: a rank updates its own neurons, microglia, astrocytes and extracellular alpha-synuclein agents. Neurons also update their own intracellular environments, including mitochondria, lysosomes, alpha-synuclein proteins and aggregates.

The ranks execute the same simulation program, but they operate on different parts of the global model. This means that a rank does not normally update all agents in the simulation. It updates only the agents assigned to its local partition.

MPI communication is required when information must cross rank boundaries. For example, if an agent or spatial effect is close to the edge of a partition, neighbouring ranks may need to exchange information in order to keep the distributed spatial state consistent.

MPI is also used to compute global quantities from local quantities. Each rank can compute values for the agents it owns, but some metrics describe the whole simulation. In these cases, each rank first computes a local contribution, and MPI combines the local values into a global value.

For an additive metric, this can be written as

$$globalMetric(t) = \sum_{rank=0}^{N-1} localMetric_{rank}(t)$$

where N is the number of MPI ranks.

For example, if each rank counts the number of healthy neurons it owns, the global number of healthy neurons is obtained by summing the local counts of all ranks.

Examples of additive metrics include:

- the number of healthy neurons;
- the number of compromised neurons;
- the number of apoptotic neurons;
- total cellular debris;
- total free alpha-synuclein;
- total aggregated alpha-synuclein.

Not every metric is combined through a simple sum. Mean values require both a global sum and a global count, while minimum and maximum values require minimum or maximum reduction operations.

This distributed organization allows the simulation to run on multiple processes. However, parallel execution is not automatically faster in every case. For small populations, the cost of MPI communication and synchronization may be greater than the computational benefit of distributing the agents. For larger simulations, distributing the work across ranks can reduce the amount of computation performed by each process.

Chapter 5

Post-Run Data Pipelines

The post-run data pipeline provides a bridge between distributed simulation execution and later analysis. During execution, the simulation records structured runtime observations. After execution, separate tools transform these observations into tables, plots and graph representations.

The simulation output is designed to record what happened during execution, while the graph pipeline interprets those records after the run. In particular, the canonical event log stores semantic simulation events for post-run analysis. It is not itself a graph format. The multilevel graphs **G0**, **G1**, **G2**, and **G3** are projections derived from the event log together with state and spatial snapshots.

This organization prevents the simulation engine from being overloaded with analysis-specific responsibilities. The engine records structured runtime observations, while separate post-processing commands transform those observations into the formats required for inspection, visualization and graph analysis.

5.1 Simulation Data Logging

The simulation framework includes a structured logging system whose purpose is to make each simulation run inspectable after execution. Since the model is both stochastic and distributed, the output of a run is not limited to final values or console messages. Instead, the framework separates different forms of runtime information according to their role in the analysis.

The current logging methodology is based on four complementary layers:

- semantic causal event logging;
- spatial snapshot logging;
- initialization logging;
- tick metric logging.

These layers do not have the same purpose. Causal event logging records *what happened* in the model and why it is relevant. Spatial snapshot logging records *where agents were* during the simulation. Initialization logging records *how the simulation started*. Tick metric logging records *how global quantities evolved* over time.

This separation is important because the same simulation can be studied from different points of view. A time-series plot requires compact scalar values, whereas causal analysis requires event records. Spatial reconstruction requires positions, while initialization inspection requires the starting configuration of agents and their ownership relations.

The logging system follows a semantic approach. Instead of directly writing low-level graph files during the simulation, the model records structured runtime events. These events are later used by the analysis and graph generation tools to reconstruct causal relations and higher-level graph representations. Each event describes a meaningful model occurrence, such as a state transition, a field modification, a degradation attempt or an aggregation process.

This approach keeps simulation logging closer to the model semantics. The simulation records events in the language of the biological and computational mechanisms, while graph construction is treated as a later interpretation of those events.

The general logic is:

1. agents execute their local rules during a simulation tick;
2. relevant model events are recorded in a semantic event log;

3. spatial positions are recorded through a separate snapshot log;
4. global scalar values are reduced across MPI ranks and written as tick metrics;
5. post-processing tools use these outputs for plots, validation, reconstruction and graph generation.

5.1.1 Runtime Log Files

During execution, the simulation produces several runtime files. These files describe semantic events, spatial states, initialization data and compact temporal metrics.

- `events_rank*.jsonl`: rank-local semantic event logs.
- `events.jsonl`: merged semantic event log, produced by rank 0 after collecting rank-local event files.
- `event_log_metadata.json`: metadata file containing the schema version and notes about the supported event types.
- `spatial_snapshots*.jsonl`: spatial state snapshots used for post-run spatial analysis and reconstruction.
- `spatial_snapshots.jsonl`: merged spatial snapshot log.
- `tick_metrics.csv`: compact scalar and count time series recorded during the simulation.
- `initialization_agents.jsonl`: initial agent population and ownership structure.
- `initialization_manifest.json`: manifest describing the initialization output.
- `initialization_summary.json`: compact summary of the initialized run.

The rank-local structure is necessary because the simulation is executed with MPI. Each rank owns only part of the global simulation state and therefore generates only the events and spatial records associated with its local agents. The merged files provide a global view of the run after execution has completed.

The analysis directory is used for outputs generated after the simulation, such as graph files, reports and mechanism summaries. Plot files are generated from tick metrics and stored separately from raw logs.

This organization keeps the simulation phase and the analysis phase distinct: the simulation emits structured observations, while post-processing transforms those observations into plots, summaries and graph representations.

5.1.2 Semantic Causal Event Logging

Semantic causal event logging records the relevant discrete events generated by the model rules. The logger stores events in JSONL format, where each line corresponds to one event.

The main event categories are:

- state transitions;
- threshold triggers;
- field changes;
- aggregation events;
- degradation events.

A **state-transition** event records a change in the internal state of an agent. For example, a neuron may become compromised, a microglial cell may become activated, or a mitochondrion may become damaged.

A **threshold-trigger** event records the fact that a perceived condition has crossed a relevant model threshold. This is useful when a transition or action depends on a field such as inflammation, debris, oxidative stress or local aggregate density.

A **field-change** event records a modification of an environmental or intracellular scalar. These events are used when an agent changes a shared field, for example by increasing inflammation, reducing debris, releasing oxidative stress or modifying intracellular energy demand.

Aggregation events describe interactions related to pathological alpha-synuclein structures. They are used when free alpha-synuclein proteins are absorbed into an aggregate, when aggregates merge or when aggregate snapshots must be recorded.

Degradation events describe lysosomal processing. They record whether a target was degraded, repaired, requeued after failure, or whether the lysosome became overwhelmed.

Event Types

The event log uses a small set of event types. Each type represents a specific class of simulation occurrence.

state_transition One agent changed biological or model state.

threshold_trigger One observed field or condition triggered a state update.

field_change One agent changed one scalar field by a signed delta.

aggregation One alpha-synuclein aggregation operation resolved.

degradation One lysosome degradation operation resolved.

These event types describe the model-level events needed for analysis. They are semantic categories: each event records a meaningful occurrence in the simulation rather than a low-level implementation detail.

Common Event Shape

Although different event types contain different contextual fields, the canonical log follows a common structure. Conceptually, each event records:

- a unique event identifier;
- the simulation tick;
- the MPI rank that produced the event;
- the event type;
- the mechanism involved;
- the actor responsible for the event;
- the effects produced by the event;
- the outcome.

The result is a log that is compact, readable and suitable for later causal analysis.

A simplified example of a field-change event is:

```
1 {
2   "event_id": "event_0_42_7",
3   "tick": 42,
4   "rank": 0,
5   "event_type": "field_change",
6   "mechanism": "microglia_debris_clearance",
7   "actor": {
8     "uid": "12:2:0",
9     "type": "Microglia",
10    "state": "Clearing"
11  },
12  "effects": [
13    {
14      "kind": "field_delta",
15      "scope": "environment",
16      "field": "extracellular_debris",
17      "delta": -0.035
18    }
19  ],
20  "outcome": "applied",
21  "context": {
22    "action": "ClearDebris"
23  }
24 }
```

Design Rule

The design rule of the event log is that each event should answer five questions:

- who acted;
- what changed;
- where and when it happened;
- which mechanism or rule caused it;
- what the outcome was.

This rule keeps the event log interpretable without turning it into a full snapshot of the simulation state. The complete state is instead reconstructed by combining event logs with spatial snapshots and metric tables when needed.

5.1.3 Spatial Snapshot Logging

Causal events describe what happened, but they do not necessarily provide a complete spatial reconstruction of the simulation. For this reason, the framework uses a separate spatial snapshot logger.

Spatial snapshot logging records the position of agents during the simulation. This is useful for reconstructing the spatial distribution of entities in the extracellular environment and in intracellular habitats.

The spatial log is intentionally separated from the semantic event log. A movement or state transition can be causally relevant, but spatial reconstruction also requires position records even when no major causal event occurs. Keeping the two logs separate avoids mixing causal semantics with geometric reconstruction data.

Each spatial snapshot record conceptually contains:

- the run identifier;
- the simulation tick;
- the MPI rank;
- the agent identifier;
- the agent class;
- the current state;
- the spatial coordinates;
- the aggregate identifier, when relevant.

5.1.4 Initialization Logging

Initialization logging records the initial conditions of the simulation. This is a distinct logging category because the initial state is not a causal event sequence. It is the starting configuration from which the dynamic process begins.

The initialization logger stores detailed information about the agents created before the first tick. This includes the initial distribution of agents across MPI ranks, their initial states, their positions and their ownership relations.

5.1.5 Tick Metric Logging

Tick metric logging records compact scalar values over time. These values summarize the global state of the simulation at each tick.

In a distributed execution, each rank computes local contributions for the agents it owns. MPI reductions are then used to obtain global values before writing the metric row. Therefore, the tick metric file describes the whole simulation, not only the rank that writes the file.

Typical metric categories include:

- extracellular scalar values;
- neuron-state counts;
- free alpha-synuclein counts;
- aggregate counts and aggregate load;

- summary quantities related to intracellular damage and pathology.

Tick metrics are designed for macroscopic analysis. They are appropriate for plotting trajectories and comparing different experimental configurations, but they are not detailed enough to explain individual transitions.

For example, a tick metric may show that the number of compromised neurons has increased. To understand which local mechanisms contributed to that change, the semantic event log must be inspected.

5.2 Graph Builder Outputs

The graph builders consume the semantic logs and spatial snapshots, then write only graph artifacts. They do not generate graph JSON files, manifest files, or report files.

The graph artifacts written by the graph pipeline are:

- `g0.gexf`;
- `g0.lite.gexf`;
- `g1.gexf`;
- `g2.gexf`;
- `g3.gexf`.

The file `g0.gexf` contains the full base graph projection. The file `g0.lite.gexf` is a lighter version intended for faster visualization. The files `g1.gexf`, `g2.gexf`, and `g3.gexf` represent the successive multilevel contractions.

This design keeps the graph generation stage focused. The graph builders do not replace the canonical logs; they derive graph projections from them.

Role in Causal and Multilevel Analysis

The logging system is the input layer of the post-run analysis pipeline. The simulation does not write graph nodes and edges directly during execution. Instead, it records semantic events, spatial snapshots, initialization data and tick-level metrics, which are later transformed into analysis artifacts.

The semantic event log is the main source for the temporal causal graph. It records state transitions, field changes, aggregation events and degradation events generated by the implemented rules. These records are used to derive causal links between agents, fields and outcomes.

The spatial snapshot log records agent positions and compartment information. It is used to reconstruct where agents were located during the run and to distinguish extracellular entities from intracellular entities associated with a specific neuron.

The initialization log stores the initial population, positions, states and ownership relations. It provides the baseline configuration against which the dynamic trace is interpreted.

The tick metric log stores compact global time series, such as extracellular scalars, neuron-state counts and alpha-synuclein aggregate counts. It is used for macroscopic inspection before moving to event-level analysis.

Together, these logs define the data flow from simulation execution to post-run reconstruction and multilevel graph generation.

Interpretation

The generated logs describe the behaviour of the implemented model. They should be read as computational traces, not as clinical measurements.

The event log identifies which model rules were triggered and what effects they produced. The spatial log records the position and compartment of selected agents. The initialization log records the starting configuration. The tick metric log summarizes the global trajectory of the run.

These outputs make the simulation reproducible, inspectable and suitable for post-processing, but any biological interpretation remains limited by the assumptions and parameter choices of the model.

Chapter 6

Multilevel Graph Analysis

This chapter describes the multilevel graph approach used by the project. The pipeline starts from the simulation logs and builds a sequence of progressively coarser directed graphs:

$$G0 \longrightarrow G1 \longrightarrow G2 \longrightarrow G3$$

The contractions applied between the levels are:

$$G0 \xrightarrow{\text{time contraction}} G1 \xrightarrow{\text{agent/state clustering}} G2 \xrightarrow{\text{topological SCC contraction}} G3$$

The implementation of the graph pipeline is organized in the following files:

```
src/analysis/graph/g0_builder.py
src/analysis/graph/multilevel_builder.py
src/analysis/schemes/time_contraction_scheme.py
src/analysis/schemes/agent_clustering_scheme.py
src/analysis/schemes/topological_scc_scheme.py
src/analysis/schemes/contraction_utils.py
```

The graph exports are written to:

output/graphs

As the following graph artifacts:

```
g0.gexf
g0.light.gexf
g1.gexf
g2.gexf
g3.gexf
```

The file `g0.light.gexf` keeps the same node identifiers and directed edges as `G0`, but removes rich node attributes and keeps only the edge relation label. It is meant for fast PyVis or dashboard visualization, not for analytical contraction.

Long provenance lists are converted into count attributes. This keeps the exported GEXF files loadable in Gephi.

Multilevel Library

The project uses the library `marco-caputo/multilevel-graphs` [1] as the main support for multilevel contractions.

The package provides the base abstractions used by the project:

- `MultilevelGraph`;
- `ContractionScheme`;
- `EdgeBasedContractionScheme`;

- `CompTable`;
- `ComponentSet`.

The project defines concrete domain-aware contraction schemes on top of these abstractions. The export path calls each scheme through `contract(...)`. When NetworkX inputs are used, the schemes rely on an adapter that materializes standard directed graphs for GEXF output.

The `TopologicalSCCContractionScheme` inherits from the library's `EdgeBasedContractionScheme`. This is relevant because G3 is not an arbitrary semantic merge. It is an edge-driven and topology-driven contraction.

6.1 G0: Temporal Causal Trace

G0 is the most detailed graph of the pipeline. It is built by `src/analysis/graph/g0_builder.py` from:

```
output/run_logs/events.jsonl
output/run_logs/spatial.snapshots.jsonl
```

Each G0 node represents a simulation entity at a specific tick. The general format is:

```
AgentNameID.State@Tick
Environment.Field@Tick
```

Examples are:

```
AlphaSynuclein_19_3_0.Misfolded@34
Neuron_112_0_0.Compromised@34
SN_dopamine_output@34
Neuron_112_0_0.oxidative_stress@34
```

Runtime decision nodes are not represented in G0. This keeps the graph focused: nodes represent agents or fields, while effects are represented as causal relations.

6.1.1 G0 Nodes

The main node categories are:

agent_state An agent in a given state at a specific tick.

aggregate An aggregate represented as an agent-like state node. It is treated as a biological object rather than as the aggregate registry itself. The registry is not a G0 node.

env_field A global substantia nigra scalar at a specific tick.

internal_field A neuron-local scalar at a specific tick.

G0 Node Attributes

- **formal_id**: canonical graph identifier;
- **display_id**: readable display identifier;
- **semantic.kind**: for example `agent_state` or `environment_field`;
- **kind**: raw log kind, such as `agent_state`, `env_field`, or `internal_field`;
- **agent_type**: biological or simulation class;
- **uid**: runtime identifier;
- **state**: agent state, when present;
- **field**: field name, when present;
- **value**: field value, when present;
- **tick**: simulation tick;
- **rank**: MPI rank;
- **run_id**: run identifier;

- **level**: raw logger level, such as **environment** or **intracellular**;
- **owner_uid**: owning neuron for intracellular agents or fields;
- **compartment**: biological compartment;
- **entity_key**: identity used for continuity contraction;
- **source_log_node_ids**: source log rows absorbed into the G0 node.

6.1.2 G0 Edges

The main edge categories are:

threshold_trigger A field threshold influences an agent transition.

$$\text{EnvironmentField@t} \longrightarrow \text{AgentState@t}$$

field_effect An agent affects an environment or intracellular field.

$$\text{AgentState@t} \longrightarrow \text{EnvironmentField@t}$$

Typical relations include **field_effect** and **internal_field_effect**.

state_transition An agent changes state.

$$\text{AgentStateA@t} \longrightarrow \text{AgentStateB@t}$$

degradation Direct agent-agent edge for lysosomal degradation.

$$\text{Lysosome} \longrightarrow \text{Target}$$

This relation is intentionally narrow: target assignment alone is not treated as degradation.

aggregation Direct agent-agent edge for alpha-synuclein aggregation.

$$\text{AlphaSynuclein} \longrightarrow \text{AlphaAggregate}$$

The aggregate registry is not represented as a graph node. Its effect is represented through edges from alpha-synuclein members into the aggregate.

continuity Temporal identity edge added by the G0 builder.

$$\text{Entity_State@t} \longrightarrow \text{Entity_State@t+1}$$

Continuity is useful because it tells later contractions which repeated nodes represent the same persistent biological or simulation entity. It can be disabled when building graphs if needed.

G0 Edge Attributes

- **edge_id**: source log edge identifier;
- **relation**: raw relation class;
- **mechanism**: biological or simulation mechanism;
- **causal_kind**: normalized causal class;
- **tick**: simulation tick;
- **rank**: MPI rank;
- **run_id**: run identifier;
- **rule_id**: rule identifier, when logged;
- **predicate**: predicate or threshold check, when logged;
- **effect_value**: numeric effect, when available;
- **effect_unit**: unit label, when present;
- **probability**: probability used by stochastic mechanisms;
- **rng_value**: sampled random value;
- **outcome**: event outcome;

- `source_kind` and `target_kind`: semantic endpoint kinds;
- `source_type` and `target_type`: endpoint agent types;
- `source_uid` and `target_uid`: endpoint identities;
- `owner_uid`: owning neuron, when applicable;
- `compartment`: biological compartment;
- `count`, `total_effect`, `mean_effect`, `first_tick`, and `last_tick`: summary values when lower edges merge.

6.2 G1: Time Contraction

G1 is built by the `TimeContractionScheme`.

The purpose of G1 is to remove raw tick repetition while preserving:

- entity identity;
- state;
- field identity;
- causal direction;
- first and last observation range.

The basic contraction rule is:

```

AgentNameID.State@t
AgentNameID.State@t+1
      ↓
AgentNameID.State

```

For fields, the analogous contraction is:

```

SN.dopamine.output@t
SN.dopamine.output@t+1
      ↓
SN.dopamine.output

```

G1 Possible Situations

Persistent same-state agent If the same agent remains in the same state across ticks, its temporal nodes are absorbed into one G1 node.

State-changing agent Different states are not merged. For example:

```

AlphaSynuclein_19_Monomer
AlphaSynuclein_19_Misfolded

```

These nodes remain distinct, usually connected by a state-transition edge.

Persistent field Repeated observations of the same field are contracted into one field node.

Windowed time contraction If `window_size` is set, time contraction is not global over the whole trace. Instead, nodes are contracted inside fixed tick windows:

```

AlphaSynuclein_19_Monomer_t0_9
AlphaSynuclein_19_Monomer_t10_19

```

This is useful when temporal windows are preferred over a fully timeless G1.

Internal continuity absorbed If continuity edges connect nodes that are contracted into the same G1 node, they are absorbed into the node's `absorbed_edge_count`.

G1 Node Attributes

G1 nodes use the shared summary attributes created by `summarize_nodes`:

- `label`: supernode label;
- `contraction`: time;
- `level`: G1;
- `semantic_kind`: common semantic kind or `mixed`;
- `agent_type`: common agent type or `mixed`;
- `uid`: common uid or `mixed`;
- `state`: common state or `mixed`;
- `field`: common field or `mixed`;
- `entity_key`: common entity key or `mixed`;
- `member_count`: number of direct lower nodes absorbed;
- `observation_count`: total lower observations represented;
- `absorbed_node_count`: recursive absorbed node count;
- `absorbed_edge_count`: lower edges absorbed inside the supernode;
- `first_seen`: first tick represented;
- `last_seen`: last tick represented;
- `original_node_ids`: source lower node identifiers.

G1 Edge Attributes

G1 edges summarize repeated lower G0 edges:

- `count`: total number of lower events represented;
- `lower_edge_count`: number of direct lower graph edges merged;
- `total_effect`: sum of represented effects;
- `mean_effect`: `total_effect` / `count`;
- `mean_of_mean_effect`: mean of lower edge means;
- `first_seen`: first tick represented;
- `last_seen`: last tick represented;
- `sign`: +, -, `state`, `structural`, or `mixed`;
- `relation`: compact relation label or `mixed`;
- `mechanism`: compact mechanism label or `mixed`;
- `causal_kind`: compact causal kind or `mixed`;
- `outcome`: compact outcome or `mixed`;
- `relations`, `mechanisms`, `causal_kinds`, and `outcomes`: provenance lists;
- `source_edge_ids`: source lower edge identifiers;
- `weight`: equal to `count`;
- `label`: compact Gephi label.

6.3 G2: Agent/State Clustering

G2 is built by the `AgentClusteringScheme`.

The purpose of G2 is to remove individual agent identity and expose class/state behaviour.

For example:

```
AlphaSynuclein.1.3.0.Misfolded
AlphaSynuclein.2.3.0.Misfolded
      ↓
AlphaSynuclein.Misfolded
```

G2 Possible Situations

Agent-state cluster All agents with the same `agent_type` and `state` are merged.

Examples are:

```
Neuron_Healthy
Neuron_Compromised
AlphaSynuclein.Misfolded
Lysosome_Active
```

Different states remain separated. This matters biologically because `Healthy`, `Compromised`, `Apoptotic`, and `Ruptured` are not equivalent.

Substantia nigra fields Substantia nigra fields remain separate singleton nodes.

Examples are:

```
SN.dopamine.output
SN.extracellular.debris
SN.inflammation.level
```

Neuron internal fields Neuron-local fields are clustered into:

```
Neuron.internal.environment
```

This prevents G2 from being dominated by one node per neuron-local field while still preserving the role of intracellular environments in causal patterns.

Mixed edge summaries If many lower edges with different mechanisms collapse into the same G2 edge, the compact label can become `mixed`. The provenance lists still retain the set of observed mechanisms, relations, and outcomes.

G2 Node Attributes

G2 uses the same core summary attributes as G1, with the addition of:

- `cluster_kind`: `agent_state_cluster` for clustered agent states, otherwise `singleton`.

G2 Edge Attributes

G2 uses the same summary edge attributes as G1, including:

- `count`;
- `lower_edge_count`;
- `total_effect`;
- `mean_effect`;
- `mean_of_mean_effect`;

- `first_seen`;
- `last_seen`;
- `sign`;
- `relation`;
- `mechanism`;
- `causal_kind`;
- `outcome`;
- provenance lists and counts;
- `weight`;
- `label`.

6.4 G3: Topological SCC Contraction

G3 is built by the `TopologicalSCCContractionScheme`.

This scheme inherits from `multilevelgraphs.contraction_schemes.EdgeBasedContractionScheme` and contracts strongly connected components of G2.

The motivation is topological rather than biological. G2 already describes which agent classes and states interact. G3 asks a different question:

Which parts of the G2 causal graph form recurring directed feedback patterns?

In a directed graph, a strongly connected component is a conservative definition of recurrence: every node in the component can reach every other node through directed paths. This avoids false positives caused by mere co-occurrence or weak connectivity.

G3 Possible Situations

feedback_component A component with more than one G2 node. This is the central G3 situation.

Interpretation:

These processes are mutually reachable and form a recurring directed causal pattern.

A conceptual example is:

`SN_inflammation_level` $\longrightarrow$ `Microglia.Activated` $\longrightarrow$ `SN_inflammation_level`

self_feedback A singleton node with an internal self-loop.

Interpretation:

The process has a self-reinforcing edge after previous contractions.

This can occur when repeated lower-level relations collapse into a self-edge at G2.

singleton_process A node that is not part of a directed feedback component.

Interpretation:

The process is present, but it is not topologically recurrent at G2.

Singletons are kept because they preserve the boundaries of feedback components. They can be sources, sinks, inputs, or downstream consequences.

Internal absorbed edge An edge whose source and target are inside the same strongly connected component is absorbed into the G3 node. These edges define the pattern's internal recurrence.

Boundary incoming edge An edge from another **G3** node into a component.

Interpretation:

External process $\longrightarrow$ recurring pattern

Boundary outgoing edge An edge from a component to another **G3** node.

Interpretation:

Recurring pattern $\longrightarrow$ downstream process

Acyclic inter-pattern edge Edges between different **G3** nodes remain as **G3** edges. These represent causal flow between feedback patterns or singleton processes.

G3 Node Attributes

G3 keeps the shared summary node attributes:

- `label`;
- `contraction: topological_scc`;
- `level: G3`;
- `semantic_kind`;
- `agent_type`;
- `uid`;
- `state`;
- `field`;
- `entity_key`;
- `member_count`;
- `observation_count`;
- `absorbed_node_count`;
- `absorbed_edge_count`;
- `first_seen`;
- `last_seen`;
- `original_node_ids`.

It also adds topological pattern attributes:

- `pattern_kind: feedback_component, self_feedback, or singleton_process`;
- `component_size`: number of **G2** nodes inside this **G3** node;
- `is_feedback_pattern`: boolean;
- `internal_edge_count`: number of direct **G2** edges inside the component;
- `boundary_in_edge_count`: number of direct **G2** edges entering the component;
- `boundary_out_edge_count`: number of direct **G2** edges leaving the component;
- `boundary_in_superedge_count`: number of incoming **G3** superedges after contraction;
- `boundary_out_superedge_count`: number of outgoing **G3** superedges after contraction;
- `internal_event_count`: sum of represented event counts on internal edges;
- `boundary_in_event_count`: sum of represented event counts on incoming boundary edges;
- `boundary_out_event_count`: sum of represented event counts on outgoing boundary edges;
- `total_internal_effect`: summed internal effect;
- `mean_internal_effect`: total internal effect divided by internal event count;
- `total_boundary_in_effect`: summed incoming boundary effect;
- `mean_boundary_in_effect`: incoming mean effect;
- `total_boundary_out_effect`: summed outgoing boundary effect;

- `mean_boundary_out_effect`: outgoing mean effect;
- `dominant_internal_relation`: most frequent internal relation weighted by event count;
- `dominant_internal_mechanism`: most frequent internal mechanism weighted by event count;
- `dominant_internal_causal_kind`: most frequent internal causal kind weighted by event count;
- `dominant_internal_sign`: dominant internal sign;
- `dominant_boundary_in_relation`: dominant incoming relation;
- `dominant_boundary_out_relation`: dominant outgoing relation;
- `node_signature`: compact count of node roles inside the pattern;
- `internal_edge_signature`: compact count of internal edge roles;
- `boundary_in_signature`: compact count of incoming edge roles;
- `boundary_out_signature`: compact count of outgoing edge roles;
- `topological_signature`: combined node and internal-edge signature;
- `component_node_ids`: lower G2 nodes represented.

For Gephi export, long provenance attributes such as `component_node_ids`, `original_node_ids`, `source_edge_ids`, and edge provenance lists are replaced with count attributes to avoid oversized GEXF fields.

G3 Edge Attributes

G3 edge attributes are produced by the same edge summarization logic used for G1 and G2. They include:

- `count`;
- `lower_edge_count`;
- `total_effect`;
- `mean_effect`;
- `mean_of_mean_effect`;
- `first_seen`;
- `last_seen`;
- `sign`;
- `relation`;
- `mechanism`;
- `causal_kind`;
- `outcome`;
- provenance list counts in Gephi exports;
- `weight`;
- `label`.

The interpretation changes slightly at G3: an edge is no longer a single direct biological relation, but a summarized causal channel between patterns.

6.5 Why SCC is Used for G3

Several topological contractions are possible, such as weak components, connected components, cliques, cycles, motifs, or strongly connected components.

Strongly connected components are the current choice because:

- the graph is directed;
- causal direction matters;
- recurrence should require mutual reachability;
- singleton non-pattern nodes remain visible;

- boundary edges make feedback inputs and outputs analyzable;
- the method is conservative and avoids merging large weakly connected regions.

Weakly connected components would be too aggressive. In a causal simulation, almost everything may be weakly connected through environment fields, causing G3 to collapse into a small number of uninformative giant components.

Cliques would be too strict. Biological causal graphs rarely contain complete directed connectivity between all process pairs, even when they are clearly part of a feedback loop.

Simple cycle enumeration would be difficult to summarize because cycles can overlap heavily. Strongly connected components provide a stable, compact supernode representation for overlapping cycles.

6.6 Practical Reading Strategy

Each graph level answers a different type of question.

Use G0 when the question is:

What exactly happened at a specific tick?

Use G1 when the question is:

How did a specific entity or state behave across time?

Use G2 when the question is:

How do biological classes and states interact in aggregate?

Use G3 when the question is:

Which recurring causal feedback patterns exist in the class/state graph?

G3 should not replace G2. It is a higher-level view intended to highlight recurrence, feedback and pattern boundaries.

Chapter 7

Dashboard Interface

The project includes a Streamlit dashboard designed to make the simulation workflow accessible without relying exclusively on command-line execution. The dashboard does not replace the command-line interface, but wraps the main project operations into an interactive environment for parameter editing, simulation monitoring, post-run inspection, spatial reconstruction and graph exploration.

7.1 Purpose of the Dashboard

The dashboard was introduced to support exploratory use of the model. Since the simulation produces heterogeneous outputs, such as YAML configurations, tick metrics, semantic event logs, initialization records, spatial snapshots and multilevel graph files, inspecting the project only through terminal commands would make the workflow harder to follow. The dashboard provides a unified interface over these artifacts.

Its main goals are:

- to edit simulation parameters before execution;
- to launch and monitor a simulation run;
- to inspect macroscopic time series after execution;
- to reconstruct the spatial distribution of agents;
- to inspect initialization records and sampled parameters;
- to build and visualize the multilevel graph hierarchy;
- to expose the project documentation inside the same interface.

7.2 Dashboard Organization

The dashboard is implemented as a multi-page Streamlit application. The entry point is `app.py`, which defines four groups of pages: *Run*, *Post-run*, *Graphs*, and *Docs*.

Group	Page	Purpose
Run	Parameters	Edit YAML configuration files
Run	Simulation	Start and monitor simulation execution
Post-run	Reconstruction	Visualize spatial snapshots by tick
Post-run	Initialization Overview	Inspect initial agents and sampled values
Post-run	Post-run Metrics	Inspect tick metrics and mechanism summaries
Graphs	Temporal Causal Graph	Explore G0
Graphs	Time-contracted Graph	Explore G1
Graphs	Agent-state Graph	Explore G2
Graphs	Topological Pattern Graph	Explore G3
Docs	Specifics and Docs	Read project documentation

7.3 Parameter Editing

The parameter page exposes the YAML files stored in the project configuration directory. The user can select a parameter file, inspect its flattened key-value representation, modify values and save the updated configuration. The dashboard validates edited values against the original value type before writing them back. This reduces the risk of introducing invalid YAML structures during interactive experimentation.

This interface is useful because the model is strongly parameter-driven. Population sizes, stochastic thresholds, degradation probabilities, transition rates, movement probabilities and scalar dynamics can be modified without changing the source code.

7.4 Simulation Monitoring

The simulation page starts the Repast4Py/MPI simulation process from the dashboard. It stores process metadata, captures console output and reads the generated tick metrics when available. This makes it possible to monitor a run while it is executing and to inspect the latest macroscopic values once data become available.

The dashboard uses the same simulation engine as the command-line workflow. Therefore, the graphical interface remains a wrapper around the canonical implementation rather than a separate execution mode.

7.5 Post-run Metrics

The metrics page reads `output/metrics/tick_metrics.csv` and displays the temporal evolution of global simulation quantities. These include extracellular debris, inflammation, dopamine output, neuron-state counts and alpha-synuclein load indicators.

The same page can also compute and load mechanism summaries from the semantic event log stored in `output/run_logs/events.jsonl`. These summaries count model-level mechanisms, such as alpha-synuclein misfolding, aggregation, lysosomal degradation, mitochondrial stress release, neuronal damage-related events and glial inflammatory responses.

7.6 Initialization and Spatial Reconstruction

The initialization page reads the initialization logs and displays the starting population structure. It can show counts by class, MPI-rank distribution, agent-specific configuration values, initial scalar values and sampled threshold distributions. This is important because stochastic initialization may produce heterogeneous agent parameters even when the same YAML file is used.

The reconstruction page reads the spatial snapshot log and allows the user to select a simulation tick, agent classes and states. The resulting view reconstructs where recorded agents were located in the extracellular grid at that time. This supports qualitative inspection of spatial patterns and helps connect temporal metrics with agent distribution.

7.7 Graph Exploration

The dashboard includes dedicated pages for the four graph levels used by the analysis pipeline. Each graph page can trigger the corresponding command-line graph builder and then load the generated GEXF file.

- **G0** represents the temporal causal graph derived from semantic events and spatial snapshots.
- **G1** applies time contraction, preserving entity and state identity while reducing temporal repetition.
- **G2** clusters agents by biological class and state, exposing aggregate interaction patterns.
- **G3** contracts strongly connected components of G2 to highlight recurring topological feedback patterns.

Graph visualization is performed through a PyVis-based component. The interface also provides tabular views of nodes, edges and metadata, allowing the user to move between visual inspection and structured graph attributes.

7.8 Role in the Analysis Workflow

The dashboard supports the complete exploratory cycle of the project. A typical workflow is:

1. edit parameters in the parameter page;
2. start a simulation from the simulation page or from the CLI;
3. inspect tick metrics and mechanism summaries;
4. reconstruct spatial snapshots and initialization data;
5. build and inspect G0, G1, G2 and G3;
6. consult the documentation page when interpreting outputs.

For this reason, the dashboard should be interpreted as an integration layer over the simulation and analysis pipeline. It improves accessibility and inspection, but the underlying model, log generation, post-processing and graph construction remain implemented in the shared project modules.

Chapter 8

Conclusions and Future Expansions

This project presented a distributed multi-agent framework for the simulation and analysis of selected mechanisms involved in Parkinson’s disease progression. The work does not aim to reproduce the full biological complexity of the disease, nor to provide a clinically predictive model. Its objective is instead methodological and exploratory: to make a selected set of pathological mechanisms explicit, executable, observable and analyzable at multiple causal levels.

The model translates biological concepts into interacting computational entities. Dopaminergic neurons are represented as macro-agents containing intracellular environments, while alpha-synuclein proteins, aggregates, mitochondria and lysosomes operate inside these local habitats. Microglia and astrocytes act in the extracellular environment, where they interact with debris, inflammation, dopamine and extracellular pathological alpha-synuclein. This organization makes it possible to connect intracellular dysfunction, neuronal damage and extracellular inflammatory feedback within a single simulation framework.

A central result of the project is the construction of an inspectable simulation infrastructure. The model does not only generate global trajectories such as dopamine decrease, inflammation increase or neuron-state evolution. It also produces semantic causal logs, spatial snapshots and temporal metrics that make the execution trace available for post-run analysis. In this sense, the simulation is not treated as a black box. The framework records what happened, where it happened, which agent was involved and which model rule produced the effect.

Another important contribution is the integration between multi-agent simulation and multilevel graph analysis. Starting from the semantic event log and the spatial snapshots, the analysis pipeline constructs a hierarchy of graphs. The base graph preserves the temporal causal trace, while the higher levels progressively remove time repetition, individual agent identity and topological detail. This produces a sequence of representations that can be read at different scales: from individual events, to recurring state interactions, to agent-class mechanisms, and finally to higher-order feedback patterns.

The main result is therefore not only a disease progression simulator, but an infrastructure for asking how a simulated progression emerges. The framework allows the user to move from local rules to global trends, and then back from global trends to the causal traces that generated them. This is particularly important for complex systems, where aggregate behaviour may hide the local interactions responsible for it.

8.1 Achieved Objectives

The initial goal of the project was to build a software environment able to configure populations and parameters, execute a distributed simulation, observe global variables, reconstruct simulation states, log causal events and generate multilevel causal graphs.

These objectives were reached through the implementation of:

- an adaptive multi-agent model based on the **see** → **next** → **action** → **do** cycle;
- a nested simulation structure separating extracellular and intracellular processes;
- agent models for neurons, microglia, astrocytes, mitochondria, lysosomes, alpha-synuclein proteins and alpha-synuclein aggregates;

- a collective aggregation mechanism for pathological alpha-synuclein, avoiding inconsistent protein-level aggregation decisions;
- distributed execution through Repast4Py and MPI;
- semantic causal logging, spatial snapshot logging and temporal metric logging;
- post-run data pipelines for table export and graph construction;
- a multilevel graph hierarchy from G0 to G3;
- dashboard and command-line tools for inspecting the generated data.

The resulting framework supports experimentation with different configurations and allows the user to inspect the simulation from both numerical and causal perspectives.

8.2 Scientific and Methodological Interpretation

The model should be interpreted as a simplified computational abstraction. The parameters used by the simulation are not calibrated biological constants. They are model parameters that control the qualitative behaviour of the implemented mechanisms.

For this reason, the conclusions obtained from simulation runs must be expressed in terms of the model assumptions. For example, if an experiment shows that reduced lysosomal efficiency increases aggregate persistence, this should be interpreted as a consequence of the implemented degradation rules, not as a quantitative biological estimate.

The value of the framework lies in its ability to make assumptions explicit. Each transition, threshold, probability and action is represented in code and can be logged during execution. This makes the model suitable for exploring mechanistic hypotheses, comparing parameter configurations and studying how local interactions can generate global behaviour.

The multilevel graph analysis further extends this interpretation. The lower levels preserve the detailed execution trace, while the higher levels expose more abstract interaction patterns. This allows the same run to be inspected from different perspectives, depending on whether the user is interested in a specific event, a concrete agent, an agent class or a recurring causal structure.

8.3 Limitations

The current model presents several limitations.

1. the biological representation is **intentionally partial**. Parkinson’s disease involves many molecular, cellular and systemic processes that are not included in the simulation. The model focuses only on a selected pathological loop involving alpha-synuclein alteration, aggregation, cellular stress, neuronal damage, debris, inflammation and glial response.
2. the model is **not quantitatively calibrated** on experimental or clinical data. The parameters are useful for controlling simulation behaviour and for performing comparative experiments, but they should not be interpreted as measured biological rates.
3. the **spatial representation is simplified**. The extracellular environment is a grid-based abstraction of a portion of the substantia nigra, and each neuron contains a simplified intracellular grid. This structure is useful for simulation and analysis, but it does not reproduce the anatomical complexity of real neural tissue.
4. the **agent behaviours are simplified**. Each biological entity is represented through a limited set of states, perceptions and actions. This makes the system understandable and executable, but it necessarily omits many biochemical pathways and regulatory mechanisms.

8.4 Future Expansions

Several future developments could improve the framework.

1. The current model could be extended with more detailed representations of neuronal vulnerability, mitochondrial dynamics, lysosomal degradation, alpha-synuclein transmission and glial signalling. Additional agent states and mechanisms could be introduced to represent more complex pathological behaviours while preserving the adaptive-agent structure.
2. The framework already supports configurable YAML parameters, but future work could introduce systematic experimental campaigns over multiple seeds and parameter ranges. This would make it possible to study robustness, sensitivity and qualitative phase changes in the simulated dynamics.
3. Larger simulations would require more careful evaluation of MPI partitioning, communication overhead, memory usage and logging volume. Future versions could include lighter logging modes, configurable snapshot frequencies and more efficient post-processing for very large event traces.

Bibliography

- [1] Marco Caputo. *MultiLevelGraphs*. <https://github.com/marco-caputo/multilevel-graphs>. 2026.
- [2] Marco Caputo, Michele Russo, and Emanuela Merelli. *Space of Data through the Lens of Multilevel Graph*. 2025. arXiv: 2503.23602 [cs.DS]. URL: <https://arxiv.org/abs/2503.23602>.
- [3] Nick Collier, Jay Ozik, and Ethan R. Tatara. *Repast4Py Documentation*. Version 3.0. Repast Project. 2025. URL: <https://repast.github.io/repast4py.site/>.
- [4] Nature Video. *Understanding Parkinson's Disease*. YouTube. URL: <https://www.youtube.com/watch?v=ckn9zybpYZ8>.
- [5] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd ed. Hoboken, NJ: John Wiley & Sons, 2009. ISBN: 978-0-470-51946-2.